AI-powered Pricing and Profitability Optimization Platform for Indian E-Commerce

PROJECT - IV (EP67006) report submitted to

Rajendra Mishra School of Engineering Entrepreneurship

in partial fulfilment of the requirements for the award of the degree of

Master of Technology

in

Engineering Entrepreneurship

by

Saharsh Agrawal

(21IM3EP27)

Under the supervision of

Professor Ram Babu Roy

Rajendra Mishra School of Engineering Entrepreneurship

Indian Institute of Technology Kharagpur

Spring Semester, 2025-26

April 30, 2026

DECLARATION

I certify that

(a) The work contained in this report has been done by me under the guidance of my supervisor.

(b) The work has not been submitted to any other Institute for any degree or diploma.

(c) I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

(d) Whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

Date: April 30, 2026

Place: Kharagpur

Saharsh Agrawal

21IM3EP27

RAJENDRA MISHRA SCHOOL OF ENGINEERING ENTREPRENEURSHIP

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

KHARAGPUR - 721302, INDIA

CERTIFICATE

This is to certify that the project report entitled “AI-powered Pricing and Profitability Optimization Platform for Indian E-Commerce” submitted by Saharsh Agrawal (Roll No. 21IM3EP27) to Indian Institute of Technology Kharagpur towards partial fulfilment of requirements for the award of degree of Master of Technology in Engineering Entrepreneurship is a record of bona fide work carried out by him under my supervision and guidance during Spring Semester, 2025-26.

Date: April 30, 2026

Place: Kharagpur

Professor Ram Babu Roy

Rajendra Mishra School Of Engineering Entrepreneurship

Indian Institute of Technology Kharagpur

Kharagpur - 721302, India

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to the individuals and institutions that have supported and guided me throughout my project in researching and working on this thesis. First and foremost, I extend my heartfelt appreciation to my thesis advisor, Professor Ram Babu Roy, for his unwavering support, expertise, and invaluable guidance. His mentorship and dedication were instrumental in shaping this research. This work would not have been possible without the collective support, guidance, and inspiration from all individuals and institutions.

Thank you.

Saharsh Agrawal

21IM3EP27

Table of Contents

Abstract

The rapid expansion of the Indian e-commerce ecosystem has intensified competition among sellers, making pricing decisions increasingly complex and critical to business success. Traditional pricing approaches, which rely on static markups or heuristic adjustments, fail to account for the dynamic interplay between demand, inventory, operational costs, and market conditions. As a result, sellers often experience suboptimal outcomes such as over-discounting, stockouts, inefficient logistics, and reduced profitability.

This thesis presents the design and development of an **AI-powered pricing and profitability optimization platform** for e-commerce sellers. The system addresses the pricing problem as an integrated decision-making challenge, combining demand forecasting, price optimization, inventory management, and cost modelling into a unified framework with the objective of maximizing **net profit**.

The proposed architecture consists of two primary components. The first component is a demand forecasting pipeline, which leverages machine learning models to estimate SKU-level demand based on historical data, product attributes, and temporal factors. The second component is an optimization engine that uses these demand estimates as input and determines optimal pricing and operational decisions by incorporating price elasticity, inventory dynamics, service-level constraints, logistics costs, and external factors such as competitor pricing and festival-driven demand variations.

A key contribution of this work is the introduction of **Inventory-Price Synchronization**, which aligns pricing decisions with supply constraints to prevent stockouts and reduce operational inefficiencies. The system also incorporates advanced features such as **festival-aware demand adjustment**, **service-level optimization through safety stock modelling**, and **order grouping for logistics efficiency**. These components enable the platform to move beyond revenue-focused strategies and instead optimize for overall profitability.

The system is implemented as a full-stack platform using modern web technologies and evaluated through a backtesting framework on the mobile phone category, selected as a proof-of-concept due to data availability and its highly dynamic market characteristics. Experimental results demonstrate that while demand-aware pricing improves revenue, integrating inventory and operational considerations leads to a significant improvement in net profit. The proposed system achieves over **25% increase in net profit** compared to baseline strategies, along with reductions in stockout rates and logistics costs.

Although the evaluation is conducted on a single product category, the framework is inherently **category-agnostic** and can be extended to other retail domains with appropriate data. The results validate the effectiveness of integrating machine learning, optimization, and operational modelling into a cohesive system for real-world e-commerce applications.

In conclusion, this thesis demonstrates that pricing in modern e-commerce environments must be treated as a multi-dimensional optimization problem. By automating complex decision-making processes and incorporating real-world constraints, the proposed platform provides a scalable and practical solution for improving profitability in competitive retail markets.

Keywords: Dynamic Pricing, Demand Forecasting, Price Optimization, Inventory Management, E-commerce Analytics, Machine Learning, Profit Maximization

Chapter 1

Introduction

1.1 Motivation

The rapid growth of e-commerce in India has fundamentally transformed the retail landscape. With the proliferation of online marketplaces such as Amazon and Flipkart, sellers now operate in an environment characterized by intense competition, dynamic consumer behavior, and high transparency in pricing. Unlike traditional retail, where pricing decisions could be relatively stable and localized, e-commerce platforms demand **continuous, data-driven pricing strategies** to remain competitive.

A defining feature of the Indian e-commerce ecosystem is its **price sensitivity**, particularly in high-volume categories such as consumer electronics and mobile phones. Consumers frequently compare prices across platforms, respond strongly to discounts, and exhibit significantly different purchasing behavior during sale events and festivals. As a result, even small changes in price can lead to large variations in demand. This makes pricing not just a tactical decision, but a **strategic lever for revenue and profitability**.

However, in practice, pricing decisions for most small and mid-sized sellers are still made using **heuristic methods**, such as fixed markups over cost, competitor matching, or manual adjustments based on intuition. These approaches fail to account for key factors such as:

- price elasticity of demand,
- inventory availability and replenishment constraints,
- logistics and holding costs,
- and temporal demand variations driven by product lifecycle and seasonal events.

This leads to suboptimal outcomes, including over-discounting, stockouts, excess inventory, and inefficient logistics—each of which directly impacts profitability.

The challenge is further compounded by the **multi-dimensional nature of the pricing problem**. A seller managing multiple SKUs across marketplaces must simultaneously consider:

- how demand responds to price changes,
- how inventory levels evolve over time,
- when to reorder and in what quantity,
- how competitors are pricing similar products,
- and how external events such as festivals affect demand.

The interaction between these factors creates a level of complexity that is difficult to manage manually, especially at scale. Traditional tools such as spreadsheets or rule-based systems are inadequate for capturing these interactions in a consistent and optimal manner.

From a technical perspective, this problem lies at the intersection of:

- **demand forecasting**,
- **optimization under uncertainty**,
- and **inventory management theory**.

While significant research exists in each of these domains individually, their integration into a unified, deployable system tailored to the Indian e-commerce context remains limited.

This thesis is motivated by the need to bridge this gap by developing an **AI-driven pricing and profitability optimization platform** that:

- leverages data to model demand behavior,
- integrates operational constraints into pricing decisions,
- and provides actionable recommendations through a user-friendly interface.

A key insight driving this work is that **maximizing revenue does not necessarily maximize profit**. Pricing strategies that aggressively increase demand may lead to stockouts, increased logistics costs, or inefficient inventory turnover. Therefore, the objective must shift from revenue maximization to **net profit optimization**, which requires a holistic view of the system.

To demonstrate the feasibility of this approach, the system is implemented and evaluated using the **mobile phone category** as a proof of concept. This category is particularly suitable due to its:

- high demand variability,
- strong price sensitivity,
- rapid product lifecycle,
- and pronounced festival-driven sales patterns.

However, the underlying methodology and system design are intended to be **category-agnostic**, enabling application across a wide range of e-commerce products.

In summary, the motivation for this thesis arises from the need to move beyond simplistic pricing heuristics and toward a **data-driven, integrated decision-making framework** that can handle the complexity of modern e-commerce operations and unlock measurable improvements in profitability.

1.2 Problem Description

The core problem addressed in this thesis is the development of a system that determines **optimal pricing and operational decisions for e-commerce sellers**, with the objective of maximizing **net profit** under realistic business constraints.

1.2.1 Nature of the Problem

In a typical e-commerce setting, a seller manages a portfolio of products, each represented as a Stock Keeping Unit (SKU). For each SKU, the seller must decide:

- the selling price,

- the inventory level to maintain,
- the timing and quantity of replenishment,
- and how to respond to external factors such as competitor pricing and seasonal demand variations.

These decisions are interdependent. The selling price influences demand, which in turn affects inventory depletion and replenishment needs. Inventory decisions impact holding costs and stockout risk, which affect realized sales and customer satisfaction. External factors such as competitor actions and festivals further complicate the demand dynamics.

The problem can therefore be viewed as a **multi-variable optimization problem**, where the objective is to maximize expected net profit over a given time horizon.

1.2.2 Key Components of the Problem

The problem involves several interacting components:

(a) Demand Uncertainty and Price Elasticity
Demand for a product is not fixed; it varies with price and other factors such as product attributes, time since launch, and market conditions. Accurately estimating this relationship is essential for making informed pricing decisions.

(b) Inventory and Replenishment Constraints
Inventory is finite and must be replenished with lead times. Holding excess inventory incurs storage costs, while insufficient inventory leads to stockouts and lost sales.

(c) Cost Structure
Profit depends not only on revenue but also on various costs, including:

- procurement cost,
- storage cost,
- logistics and transportation cost,
- and potential penalties associated with stockouts.

(d) External Market Signals
Competitor pricing and marketplace dynamics influence consumer choice and must be considered when setting prices.

(e) Temporal and Event-Based Variations
Demand is affected by time-dependent factors such as:

- product lifecycle (e.g., hype decay),
- seasonal trends,
- and festival-driven demand spikes.

1.2.3 Mathematical Formulation

The objective is to determine decision variables such as price and service level for each SKU over time, such that the expected net profit is maximized.

At a high level, the problem can be expressed as:

$$\max \quad \text{Net Profit} = \text{Revenue} - \text{Total Costs}$$

where:

- Revenue depends on price and demand,
- Demand is a function of price and contextual factors,
- Total Costs include holding, logistics, and stockout-related costs.

The challenge lies in the fact that:

- demand is uncertain and price-dependent,
- costs are dynamic and interdependent,
- and decisions must be made simultaneously across multiple SKUs and time periods.

1.2.4 Limitations of Existing Approaches

Existing pricing approaches in practice suffer from several limitations:

- **Static Pricing:** Fixed markup strategies ignore demand elasticity and market dynamics.
- **Competitor Matching:** Reactive pricing based on competitors may erode margins without guaranteeing increased sales.
- **Demand-Only Optimization:** Systems that optimize price based only on demand forecasts may increase revenue but fail to account for inventory and operational costs.
- **Manual Decision-Making:** Human decision-makers are unable to consistently process the large number of variables involved.

These limitations highlight the need for a more integrated approach that simultaneously considers demand, cost, and operational constraints.

1.2.5 Scope of the Proposed Solution

This thesis proposes a system that addresses the above challenges through a **two-pipeline architecture**:

- **Pipeline 1:** Generates demand forecasts using machine learning models.
- **Pipeline 2:** Uses these forecasts as input to an optimization engine that determines pricing and operational decisions.

The system incorporates:

- price-demand relationships,
- inventory and service-level modelling,
- logistics and cost considerations,
- competitor and festival effects.

The solution is implemented as a **full-stack platform**, enabling not only computation but also visualization and user interaction.

1.2.6 Proof-of-Concept Scope

Due to data constraints, the system is demonstrated using the **mobile phone category**. This serves as a proof-of-concept to validate the methodology and system design. The choice of this category is driven by data availability and its relevance to dynamic pricing scenarios.

However, it is important to emphasize that:

- the framework is **not limited to mobile phones**,
- the same methodology can be extended to other categories with appropriate data.

1.2.7 Summary of the Problem

In summary, the problem addressed in this thesis is:

To design and implement a system that determines optimal pricing and operational decisions for e-commerce products by integrating demand forecasting, inventory management, and cost optimization, with the objective of maximizing net profit under realistic constraints.

This problem forms the foundation for the system developed and evaluated in the subsequent chapters.

1.3 Literature Review

The problem of pricing and profitability optimization in retail and e-commerce lies at the intersection of demand forecasting, econometric modelling, optimization theory, and artificial intelligence. Over the years, researchers have explored these domains independently; however, their integration into a unified, operationally aware system remains an active area of research.

1.3.1 Demand Forecasting in Retail Systems

Demand forecasting forms the foundational layer of any intelligent pricing system. Traditional approaches rely heavily on time-series models, such as ARIMA and exponential smoothing, to extrapolate historical demand patterns. However, these classical models often struggle to effectively incorporate exogenous variables, such as dynamic price changes, promotional events, and shifting product attributes.

With the increasing availability of granular retail data, machine learning-based approaches have largely superseded classical models. A systematic review by Chowdhury et al. (2025) highlights that

ensemble learning methods (e.g., Random Forest, Gradient Boosting) and neural networks significantly outperform statistical baselines in retail demand forecasting, particularly in high-variance environments like e-commerce. Studies such as Hussain and Patel (2024) emphasize the role of predictive data analytics in improving operational efficiency, demonstrating how the integration of transactional, behavioral, and contextual data leads to superior demand estimation. Similarly, advanced machine learning architectures have proven effective at capturing the complex, nonlinear relationships between demand drivers, enabling more robust predictions in volatile market systems (Chen et al., 2024).

Recent literature has also shifted toward deep learning and hybrid models to handle cold-start problems and data sparsity. For example, Gupta and Rao (2024) demonstrate that neural network-based architectures can accurately map demand patterns even under conditions of limited historical price variation, yielding realistic elasticity estimates and high predictive performance. Despite these advances, demand forecasting alone does not solve the pricing problem; forecasts must be translated into actionable strategies through robust optimization frameworks.

1.3.2 Price Elasticity and Demand Modelling

A critical bridge between forecasting and optimization is the price elasticity of demand, which quantifies consumer responsiveness to price changes. Classical econometric approaches model this relationship using linear or log-linear demand functions. While these models offer high interpretability, they frequently fail to capture the complex, nonlinear interactions and shifting consumer thresholds present in modern e-commerce data.

Recent research has pivoted toward data-driven elasticity estimation, where machine learning models implicitly learn demand-price curves. Mishra and Verma (2023) highlight the importance of behavioral modelling in capturing consumer response patterns, noting that elasticity is highly contextual and heavily influenced by the online marketplace environment. Furthermore, studies in cognitive computing and intelligent systems (Lee & Wang, 2023) demonstrate that integrating predictive modelling with contextual reasoning allows systems to infer elasticity dynamically, aligning with the need for pricing engines that respond instantly to changing market conditions.

However, a recurring limitation in the existing literature is that elasticity estimation is often treated as an isolated analytical exercise, divorced from the operational constraints (e.g., inventory limits, logistics costs) necessary for real-world deployment.

1.3.3 Optimization Models for Pricing and Operations

Optimization theory provides the mathematical scaffolding required to translate demand insights into actionable pricing decisions. Classical models focus on profit maximization formulated as constrained, nonlinear optimization problems.

Nair and Kumar (2023) provide a comprehensive overview of mathematical optimization techniques utilized in the decision sciences, detailing constrained and multi-objective formulations. These mathematical models are particularly critical in retail, where multiple competing factors—such as procurement costs, holding costs, demand velocity, and competitor pricing—must be balanced simultaneously.

Recent studies highlight the efficacy of AI-driven optimization frameworks in industrial and retail decision-making. Deshmukh and Sharma (2024) demonstrate how fusing machine learning outputs with deterministic optimization models enables scalable and adaptive decision-support systems. Furthermore, heuristic and metaheuristic approaches, such as genetic algorithms and adaptive search parameters (Agarwal, 2023), have been successfully deployed to solve complex optimization problems where exact analytical solutions are computationally prohibitive. These techniques are highly advantageous when navigating the high-dimensional search spaces required for portfolio-wide SKU optimization.

1.3.4 Reinforcement Learning and Dynamic Pricing

Dynamic pricing has been extensively studied through the lens of reinforcement learning (RL). RL frameworks treat pricing as a sequential decision-making process, wherein an autonomous agent learns optimal pricing policies through continuous interaction with market environments.

Kumar and Singh (2019) provide a foundational survey of deep reinforcement learning methods, highlighting their transformative potential in dynamic pricing and inventory control. More recent work by Gupta and Rao (2024) explores advanced RL frameworks for intelligent systems, showcasing their ability to adapt to dynamic environments with delayed rewards.

While RL offers powerful theoretical advantages—most notably its capacity to model long-term profitability and adapt without explicit programmatic rules—it presents significant practical hurdles. These include vast data requirements, a lack of model interpretability ("black-box" pricing), and high implementation risks in live retail environments. Consequently, practical commercial systems generally favor hybrid approaches that combine deterministic optimization with predictive machine learning.

1.3.5 Integrated Decision Systems and AI Platforms

The forefront of operations research is currently focused on integrated decision systems, where forecasting, econometric modelling, and operational logic merge into a unified architecture. Lee and Wang (2023) describe cognitive decision-support platforms that aggregate various AI microservices to augment human decision-making. Similarly, Chen et al. (2024) emphasize the necessity of end-to-end intelligent systems for managing complex logistical and operational environments.

These studies yield a crucial insight: the true commercial value of artificial intelligence lies not in isolated predictive models, but in their orchestration into coherent systems that directly support real-world, constrained decision-making.

1.3.6 Research Gap

Despite significant advances across individual analytical domains, several critical gaps persist in the literature:

- **Lack of Integration:** Most studies focus entirely on either demand forecasting or mathematical pricing optimization, failing to integrate both into a unified, closed-loop system.

- **Limited Operational Awareness:** Existing pricing models frequently ignore physical supply chain constraints, such as inventory depletion rates, logistics overhead, and safety stock service levels.
- **Absence of Real-World Deployment Focus:** Many theoretical models are evaluated in academic isolation, completely bypassing the practical realities of e-commerce platform constraints and UI/UX explainability requirements.
- **Insufficient Focus on the Indian E-commerce Context:** The unique structural characteristics of the Indian market—such as hyper-price sensitivity, immense festival-driven demand spikes, and complex multi-marketplace competition—remain severely underexplored.

This thesis directly addresses these gaps by developing an integrated, end-to-end pricing and profitability optimization platform that synthesizes demand forecasting, econometric elasticity modelling, and operationally constrained decision-making, specifically tailored for the Indian e-commerce landscape.

1.4 Related Products and Market Gap

1.4.1 Overview of Existing Solutions

Table 1.4.1 compares leading pricing tools:

Revionics (now Aptos) is an enterprise AI pricing suite for large retailers, offering base pricing, promotions, and markdown optimization.

PROS provides an AI-driven price & offer management platform (especially B2B/SAP users) with real-time pricing insights.

Pricefx is a cloud-native pricing SaaS for mid-large firms (price optimization, quoting, CPQ).

Feedvisor (Amazon) combines dynamic repricing and ad optimization in one platform.

Competera is an AI retail-pricing tool focusing on competitive data and elasticity-based repricing.

On the smaller end, there are numerous Amazon/Flipkart repricers: eVanik and PriceKaZam claim AI repricing and inventory alerts for India, while RepricerExpress or SellerSnap target mid-size sellers on Amazon/eBay with rules-based repricing and margin protection. ChannelAdvisor and others offer multi-channel listing/pricing modules.

However, most of these either:

(a) lack integrated demand/inventory logic,

or (b) focus on marketplaces rather than retail catalogs,

or (c) do not support event planning or service-level tradeoffs.

Pricing models range from enterprise subscriptions (Revionics/PROS/Pricefx) to fixed-tier per-SKU plans (Amazon repricers) — but detailed pricing is rarely public.

Feature Gap Analysis: No single product offers the combination of features in MTP-II.

Table 1.4.1 (below) summarizes each vendor vs. our platform. For example, Revionics/PROS lack marketplace competitor scraping and festival modules. Amazon repricers lack multi-SKU inventory syncing and do not forecast demand beyond the Buy Box. Competera/Pricefx focus on price data but do not handle service-level or order grouping.

Our platform uniquely integrates ML demand forecasts, elasticity, competitor monitoring, and inventory-service modelling in one workflow.

Feature	Revionics	PROS	Pricefx	Feedvisor	Competera	RepricerExpress/ Snap	Our Product
Demand forecasting	–	–	–	–	–	–	✓ (ML/AI)
Elasticity estimation	–	–	–	–	–	–	✓ (data-driven)
Inventory & service-level	–	–	–	–	–	–	✓
Festival/event modelling	–	–	–	–	–	–	✓ (multipliers)
Competitor price scraping	–	–	–	✓	✓	✓	✓
Automatic repricing	–	–	✓	✓	✓	✓	✓
Order grouping (logistics)	–	–	–	–	–	–	✓
Dashboard UI & simulation	✓	✓	✓	✓	✓	✓	✓
Multi-tenant SaaS	✓	✓	✓	✓	✓	✓	✓

Table 1.4.1: *Feature comparison of pricing platforms. ✓= supported; – = not supported. (Feature support inferred from vendor info).*

1.4.2 Limitations of Existing Products

Despite the availability of these solutions, several limitations persist:

1. Enterprise-Centric Design
Most advanced platforms are designed for large enterprises and require significant financial investment, making them inaccessible to small and mid-sized sellers.

2. Pricing-Centric Focus
Many tools focus primarily on pricing, without integrating inventory management, logistics optimization, or service-level considerations.

3. Limited Customization and Transparency
Black-box models often lack interpretability, making it difficult for users to trust or understand recommendations.

4. Lack of Contextual Adaptation
Existing systems may not fully account for region-specific factors such as:

- festival-driven demand in India,
- high price sensitivity,
- and marketplace-specific dynamics.

5. Reactive Rather than Proactive Approaches
Rule-based repricing tools react to competitor changes rather than proactively optimizing for profit.

1.4.3 Market Gap

The analysis of existing products reveals a clear gap in the market:

There is a lack of **affordable, integrated, and intelligent pricing systems** tailored for mid-market e-commerce sellers, particularly in emerging markets such as India.

Specifically, current solutions fail to provide:

- **end-to-end integration** of demand forecasting, pricing, inventory, and logistics,
- **profit-focused optimization** rather than revenue or price matching,
- **interpretability and usability** for non-technical users,
- and **adaptability to local market conditions**.

1.4.4 Positioning of the Proposed System

The system developed in this thesis is designed to address these gaps by providing:

- A **holistic optimization framework** that integrates pricing with inventory and operations
- A focus on **net profit maximization** rather than isolated metrics
- A **modular architecture** combining demand forecasting and optimization
- A **user-facing platform** with actionable insights and scenario simulation

Unlike enterprise solutions, the proposed system aims to be **scalable, cost-effective, and accessible to mid-market sellers**.

Chapter 2

System Overview

2.1 Introduction and Product Features

Modern Indian e-commerce operates in a highly competitive environment where pricing decisions affect not only revenue, but also inventory movement, logistics efficiency, and long-term profitability. Sellers are expected to respond to frequent demand shifts caused by seasonality, festive events, marketplace competition, changing consumer sensitivity, and variable operational costs. In such a setting, manual pricing strategies based on intuition, flat markups, or competitor imitation are often too coarse to preserve net profit consistently. The system developed in this thesis addresses this problem by moving from isolated price selection toward a broader AI-driven profitability platform that integrates demand-aware pricing, inventory control, competitor awareness, and event-sensitive decision-making. The system is designed as a practical product prototype for Indian e-commerce sellers, while remaining generic enough to be extended beyond the demonstration category used in this thesis.

The current implementation is organized as a modular platform rather than a single pricing model. At the center is a pricing and optimization engine that computes net-profit-maximizing decisions under business constraints. Around this core are supporting modules for competitor intelligence, inventory monitoring, festival/event handling, service-level optimization, order grouping, and dashboard-based decision support. The architecture is intentionally designed to be product-like: a seller can upload or connect data, inspect recommendations, simulate changes, and apply the resulting actions through an interactive interface. For the present MTP-II demonstration, the complete system is evaluated on a mobile phone category, but the platform itself is category-agnostic and can be applied to any SKU family once sufficient live data is available.

The core proposition of the system is that **net profit can be optimized more effectively when pricing, inventory, logistics, and service-level decisions are treated jointly rather than independently**. In contrast to MTP-I, which focused primarily on demand-aware price optimization, MTP-II incorporates additional operational variables such as storage cost, transport cost, holding cost, reorder timing, competitor signals, and seasonal multipliers. This makes the platform closer to a real deployment setting and also strengthens its product value proposition.

2.1.1 Core Product Proposition

The platform learns how demand responds to price and then translates that information into actionable recommendations for pricing, stock planning, and profit protection. The recommendation is not limited to a single “best price” number; instead, the system can return an optimal price, a feasible price band, an expected net profit impact, a service-level recommendation, and operational alerts related to understocking, overpricing, and competitor pressure. The system therefore functions both as a pricing engine and as a profitability orchestration layer.

2.1.2 Key Product Features

The implemented system supports the following major capabilities: demand-aware pricing using an external forecasting pipeline; net-profit maximization under margin, inventory, and logistics constraints; service level as a decision variable in optimization; safety stock and reorder-point computation from demand variance and lead-time assumptions; competitor-aware guardrails using stored or live market data; festival-aware demand adjustment using category-specific multipliers; order grouping to reduce logistics cost; lifecycle-aware price recovery and hype-decay analysis; and an interactive web dashboard for simulation and decision support. These modules are already incorporated into the current system architecture and are not merely conceptual future work.

Module	Primary Inputs	Primary Outputs	Role in System
Demand Forecasting (Pipeline 1)	SKU attributes, Price (P), Month, Competitor Price	Normalized Weekly Demand Mean (μ_D) & Variance (σ^2_D)	Serves as the "Intelligence Layer" providing baseline market volume potential without seasonal noise.
Price Optimization Engine (Pipeline 2)	Normalized Demand, Cost structure, Margin constraints	Absolute Optimal Price (P^*), Profit-Price Curves	The "Action Layer" that executes a 25-node grid search to maximize net total profit.
Festival/Event Handler	Normalized Demand, Indian Festival Calendar, SKU sensitivity	Festival-Augmented Expected Demand ($D_{estimated}$)	Dynamically scales demand using statistical multipliers (1.1x–1.45x) for Indian sale events.
Inventory Orchestrator	Demand Variance, Target Service Level (SL), Lead Time, On-hand stock	Safety Stock (SS), Reorder Point (ROP), Throttling Price	Syncs pricing with supply chain to prevent search-rank killing stockouts through demand throttling.
Logistics Efficiency Module	Demand Velocity, Portfolio ROPs, Transport costs	Optimized Grouped Order Schedules	Minimizes "Net Logistics Drain" by aligning replenishment across multiple SKUs into grouped shipments.
Product Lifecycle (Hype Decay)	Launch Proxy date, log_days_since_launch	Continuous price-ceiling micro-adjustments	Automates "Price Recovery" and depreciation tracking to capture surplus as SKU hype decays.
Competitor Intelligence	Stored/Live Competitor Feeds, SKU Elasticity	Undercut Frequency, Risk Markers (High/Med/Low)	Monitors market parity to ensure recommendations remain competitive

			within category guardrails.
Web Dashboard & AI Simulator	Optimization outputs, User decision variables (SL, Festival toggles)	KPI cards (Recoverable Missed Profit), "What-If" visualizations	Provides "Glass-Box" transparency and human-in-the-loop decision support for the seller.

***Table 2.1**: System Module Definitions and Functional Roles*

2.2 System Architecture Overview

The system follows a layered modular architecture that separates data acquisition, forecasting, optimization, and presentation. This separation improves interpretability, makes the platform easier to extend, and mirrors how a production SaaS product would be organized. The architecture is based on a multi-tenant backend with authenticated seller contexts, a service layer that performs the mathematical computation, and a frontend layer that exposes recommendations and simulations through dashboards and action-oriented widgets. The system currently uses a React/Next.js frontend, a FastAPI backend, and MongoDB for storage.

At the highest level, the architecture can be understood as a pipeline that accepts seller and market inputs, processes them through a forecasting layer, then passes the forecast output into a constrained optimization engine. That engine computes prices, service levels, safety stock, and action recommendations while accounting for operational costs and category-specific effects. The output is then exposed to the user through visual analytics, scenario comparison, and actionable tables. This architecture is suitable both for research evaluation and for a future commercial deployment.

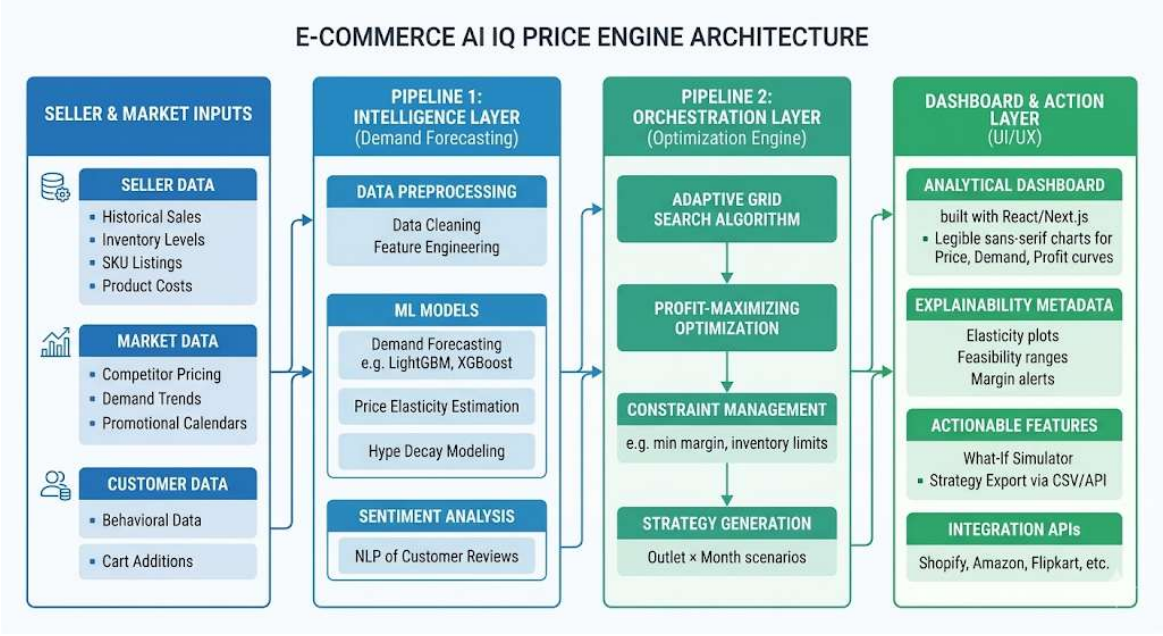

***Figure 2.2**: System Architecture Diagram*

2.2.1 Pipeline 1: Demand Forecasting Framework

Pipeline 1, developed by Shashwat Kushwaha, is treated in this thesis as a black-box forecasting service that provides demand estimates used by the optimization engine. Its purpose is to predict SKU-level demand in the Indian e-commerce context, particularly for mobile phones in the current proof-of-concept setting. The forecasting pipeline builds a unified analytical representation from heterogeneous sources and produces demand-related outputs that are later consumed by Pipeline 2. In the present thesis, the internal details of this pipeline are not the main contribution, but its inputs, outputs, and role in the system must be clearly understood because the optimization layer depends on it.

At a high level, Pipeline 1 uses historical transaction data together with product attributes, customer review signals, and calendar-based seasonal information to forecast demand at the SKU level. The methodology includes feature engineering such as hype-decay variables, cyclic month encoding, and sentiment extraction from reviews; it also includes machine learning models such as LightGBM and XGBoost, along with a cold-start track using a two-stage approach for new SKUs. The demand output is normalized so that seasonal festival effects from past years are removed, which allows Pipeline 2 to reintroduce category-specific festival multipliers explicitly in the optimization stage. This design prevents double counting of festival effects and makes the final demand estimate more transparent.

The key output of Pipeline 1 is a demand estimate that can be interpreted as the base demand level for a given SKU, market condition, and price point. In this thesis, that demand is used as the input to price optimization, inventory planning, and scenario simulation. Therefore, Pipeline 1 acts as the "engine fuel" for the rest of the platform. Its quality directly affects the quality of every downstream recommendation, especially price, reorder point, and service-level suggestions.

2.2.2 Pipeline 2: Profitability and Price Optimization Engine

Pipeline 2 is the core contribution of this thesis. It converts forecasted demand into concrete business decisions by maximizing net profit under realistic constraints. In contrast to the earlier MTP-I formulation, the current engine is not restricted to pricing alone. It now jointly considers price, service level, safety stock, holding cost, transport cost, competitor positioning, and category-specific event effects. This is what turns the system from a narrow optimization tool into a broader profitability platform.

The optimization problem can be written as a constrained maximization over product, time, and marketplace dimensions. Let $s$denote SKU, $m$denote month or time period, and $o$denote outlet or marketplace. The decision variables include the recommended price $p_{s,m,o}$and the service level $SL_{s,m,o}$. A simplified formulation is given below:

$$\max_{\{p_{s,m,o},SL_{s,m,o}\}} \sum_{s,m,o} [\, p_{s,m,o}\, \widehat{D}_{s,m,o}(p_{s,m,o}) - C^{\text{cogs}}_{s,m,o}\, \widehat{D}_{s,m,o}(p_{s,m,o}) - C^{\text{hold}}_{s,m,o} - C^{\text{log}}_{s,m,o} - C^{\text{stockout}}_{s,m,o}]$$

subject to feasible price bounds, minimum margin requirements, non-negative demand, inventory feasibility, and operational constraints on service level and replenishment. Here, $\widehat{D}_{s,m,o}(p)$is the forecasted demand returned by Pipeline 1 after adjustment for platform-specific multipliers and scenario-specific logic.

The net profit objective is important because it captures the true business effect of the recommendation. A price that increases revenue is not necessarily desirable if it causes excessive stockouts, higher holding costs, or higher logistics overhead. By optimizing net profit rather than gross revenue alone, the engine can choose a slightly lower volume strategy when that yields a larger overall return, or a slightly higher price when preserving inventory and logistics efficiency improves total profit. This is the correct objective for an operational pricing platform.

2.2.3 Supporting Modules Integrated into the Platform

The pricing engine is supported by several modules that extend the platform beyond pure price optimization. The inventory module computes safety stock and reorder points using service level and demand variance. The competitor module stores historic competitor prices for current analysis and supports future live-data integration. The festival module uses category-specific festival multipliers to adjust normalized base demand for upcoming events. The order grouping module reduces logistics cost by consolidating replenishment decisions across SKUs when timing allows. The lifecycle module supports price recovery and hype-decay analysis, which is particularly important for fast-moving product categories such as mobile phones.

For inventory planning, the system uses service level as an optimization variable rather than as a fixed hard-coded setting. The safety stock buffer is derived from the chosen service level and demand variability. A standard formulation is:

$$SS_{s,m,o} = z(SL_{s,m,o}) \cdot \sigma_{d,s,m,o} \cdot \sqrt{L_{s,o}}$$

$$ROP_{s,m,o} = \mu_{d,s,m,o} \cdot L_{s,o} + SS_{s,m,o}$$

where $SS_{s,m,o}$is the safety stock, $ROP_{s,m,o}$is the reorder point, $z(SL_{s,m,o})$is the service-level multiplier, $\sigma_{d,s,m,o}$is the demand standard deviation from the forecast distribution, and $L_{s,o}$is the replenishment lead time. This formulation allows the engine to balance stockout risk against holding cost and to select a service level that maximizes net profit rather than minimizing stockouts blindly.

2.3 Data Flow and Workflow

The platform follows a clean operational flow from seller input to final recommendation. The seller provides product, cost, inventory, and marketplace data, and the platform either ingests or references competitor information and festival calendars. Pipeline 1 generates normalized base demand estimates. Pipeline 2 then applies festival multipliers, considers market context, and evaluates candidate prices and service levels under profit constraints. The output is a recommendation package containing optimal price, feasible price range, expected profit impact, reorder guidance, and warning signals such as understocking or overpricing.

The workflow is designed so that the user does not interact with raw model internals. Instead, the platform returns business-readable outputs: suggested price, margin-safe band, expected units sold, expected net profit, inventory implication, and comparison against alternative scenarios. This is an important product feature because it reduces the barrier between technical optimization and seller

adoption. A seller can understand not only what the engine recommends, but also why the recommendation is reasonable.

The demand adjustment logic is particularly important for festival periods. Pipeline 1 provides a normalized base demand that is free from historical festival distortion, and Pipeline 2 multiplies that by SKU-specific festival multipliers derived statistically for the current category. This allows the system to reflect the elevated demand environment of events such as Diwali or major e-commerce sale periods without contaminating the underlying demand model. The same approach is also useful for post-festival price recovery, where demand elasticity gradually decays back toward the baseline and the optimizer can gradually raise prices to restore margin.

The platform also supports a cold-start phase for new users or new SKUs. During this phase, the system collects live transaction history and competitor observations for a few months before relying heavily on optimization outputs. This is necessary because pricing recommendations become more reliable as the platform observes actual seller-specific demand and market behavior. In that sense, the platform is designed not only as an optimizer, but also as a data accumulation and learning system that improves as the retailer's own operational history grows.

2.3.1 Scope of the Thesis

The present thesis focuses on the architecture, implementation, and evaluation of the profit-optimization platform, while treating the forecasting pipeline as an external input service. The platform is demonstrated on mobile phones because sufficiently granular public data is available for that category, but the architecture is deliberately category-agnostic. Therefore, the mobile phone use case should be read as a proof of concept for broader deployment across product categories once live retailer data becomes available.

Chapter 3

Demand Modelling Framework (Pipeline 1 Overview)

3.1 Introduction

The pricing and profitability platform developed in this thesis depends on a reliable demand estimation layer. Since price recommendations are only meaningful when demand response is known or reasonably approximated, the forecasting pipeline forms the analytical foundation for the optimization engine. In this system, demand modelling is not treated as a separate end-user product feature; rather, it is an enabling service that supplies the optimization engine with normalized demand estimates, elasticity signals, and scenario-specific predictions. The forecasting layer is therefore essential to the entire platform, even though the core thesis contribution lies in the downstream optimization and profitability orchestration logic.

The demand pipeline used in this work is treated as a black-box model in the present thesis. Its internal implementation belongs to the collaborator's work, but its output is integrated directly into the optimization layer described in Chapter 4. For the purposes of this thesis, it is sufficient to understand the forecasting system in terms of its inputs, modelling logic, and outputs. The forecasting pipeline was designed for the Indian mobile-phone category in the proof-of-concept setting used here, but the general formulation is reusable for other categories as well.

3.2 Problem Setting

The demand modelling problem can be stated as follows. Given historical sales records and product-level attributes, estimate the demand response of a SKU under different market conditions, including price variation, seasonality, product lifecycle stage, and event-driven effects. The output should be sufficiently structured so that it can later be used by the price optimization layer to compute profit-maximizing recommendations. In this thesis, the demand estimate is not used only for forecasting accuracy evaluation; it is used operationally as an input to pricing, service-level planning, and inventory decisions.

Because the platform is intended for real seller deployment, the forecasting system must be robust to practical issues such as missing data, changing product attributes, and the cold-start problem for new SKUs. In the current demonstration setup, these issues are handled through a combination of feature engineering, model choice, and later-stage normalization. The aim is not to build an abstract forecasting benchmark, but to create a useful demand layer that can support a profitable pricing product in the Indian e-commerce context.

3.3 Data Representation and Feature Construction

The forecasting pipeline builds a structured representation of each SKU using heterogeneous signals. The data foundation combines transactional sales information, product catalog attributes, customer review signals, and Indian festival/calendar information into a unified analytical file. This design is important because mobile-phone demand is shaped not only by price, but also by lifecycle effects, consumer perception, and event-based purchase spikes.

The feature engineering process includes several components that capture different parts of the demand process. First, the pipeline includes a **hype-decay** variable based on the number of days since launch, which reflects the fact that mobile-phone demand typically weakens as a product ages. Second, cyclic time encoding is used to represent month-level seasonality in a mathematically smooth way. Third, review sentiment is used as a proxy for customer perception and product acceptance. Fourth, festival or event indicators are incorporated to capture abrupt demand increases around sale windows. These features make the model more suitable for the Indian e-commerce environment, where calendar effects and product lifecycle effects are both material.

Feature Type	Example Feature	Role in Demand Modelling
Pricing Signals	Log-transformed Price (log_price), Discount %	Acts as the primary determinant of demand; used as the core variable to fit $log-log$ demand curves and estimate price elasticity.
Hardware Specifications	RAM capacity (ram_gb), Internal Storage ($storage\_gb$)	Distinguishes between product tiers (budget vs. flagship) and accounts for hardware-driven demand variation.
Temporal Dynamics	Cyclic Month Encoding ($month\_sin/cos$), Log days since launch	Preserves calendar continuity to model regular seasonality and provides a "hype-decay" proxy for mobile phone lifecycles.
Event-Driven Features	Binary Festive Flag ($is\_festive$), Festival Multiplier	Captures punctuated demand surges caused by major Indian e-commerce events like Diwali or Big Billion Days.
Customer Sentiment	Mean VADER compound score ($sentiment\_avg$), Review Velocity	Integrates qualitative qualitative signals from user feedback and accumulated reputation as a proxy for demand momentum.
Market Categorization	Manufacturer Brand, Sales Channel (Online vs. Offline)	Captures brand-loyalty effects and identifies structural demand differences between digital and physical retail channels.

Table 3.3: *Feature Engineering for E-Commerce Demand Modelling*

3.4 Modelling Approach

The demand layer uses supervised machine learning to approximate the relationship between price and expected sales. The main modelling track uses gradient-boosting methods such as LightGBM and XGBoost, tuned through hyperparameter search to reduce forecast error. It includes a separate cold-start track designed to infer demand behavior for products with limited or no sales history. This dual-

track setup is useful because a commercial pricing platform must support both mature SKUs and newly listed products.

The central output of the demand model is a forecast of expected sales for a given SKU under a chosen price and context. In operational terms, the model is used to answer questions of the form: if the seller changes the price, what happens to expected demand? This is the key input needed for the next optimization stage, because profit cannot be optimized without estimating how quantity sold changes as price changes. For the present thesis, the model is used as a forecast service rather than as a standalone research object.

The overall logic can be represented as:

$$\widehat{D}_{s,m,o} = f(X_{s,m,o})$$

where $\widehat{D}_{s,m,o}$ is the predicted demand for SKU s, month m, and outlet or marketplace o, and $X_{s,m,o}$ is the vector of explanatory features including price, lifecycle stage, month, sentiment, and event-related variables.

3.5 Normalized Demand and Festival Separation

A key design choice in MTP-II is that the forecasting pipeline returns **normalized base demand**, meaning the historical festival effect has been removed from the base signal. This is important because the optimization layer applies its own festival multipliers later, based on current category-specific festival assumptions. If festival effects were left inside the base demand and then multiplied again downstream, the system would double-count the same seasonal effect and overstate demand. The normalization step therefore keeps the two layers logically separated and mathematically cleaner.

In the MTP-II system, the final demand used for pricing and inventory decisions can be expressed conceptually as:

$$\widehat{D}^{\text{final}}_{s,m,o} = \widehat{D}^{\text{base}}_{s,m,o} \times F_{s,m}$$

where $\widehat{D}^{\text{base}}_{s,m,o}$ is the normalized demand from Pipeline 1 and $F_{s,m}$ is the category- and SKU-specific festival multiplier applied in Pipeline 2. This separation allows the platform to model current-year event effects more explicitly, which is especially useful for the demonstration category used in this thesis.

3.6 Cold-Start Handling

A common issue in commercial pricing systems is the cold-start problem, where a new SKU has too little sales history to support a reliable demand curve. The platform addresses this by first entering a data-collection phase for newly onboarded retailer catalogs. During this period, the system gathers live transaction data and competitor information from online sources. Once enough behavior-specific history has been accumulated, the platform begins using the learned demand patterns for reliable pricing projection. This makes the system usable in practice even when historical data is initially sparse.

3.7 Forecast Outputs and Their Role in Optimization

The forecasting layer produces outputs that are directly used by the optimization engine. The most important output is the base demand estimate under a given price and market context. Depending on the implementation detail exposed through the interface, the forecast output may also include demand variance or a distributional summary that is used later for service-level and safety-stock calculations. In the full platform, these outputs support not only pricing but also inventory decisions and reorder timing.

Operationally, this means the demand model serves three downstream functions. First, it allows the optimizer to compute profit as a function of price. Second, it informs the safety stock and reorder point calculation through demand variability and lead-time logic. Third, it supports scenario analysis, such as festival simulations or price-recovery simulations. This makes the forecasting layer a shared analytical resource for the full platform rather than a one-time preprocessing step.

3.8 Key Modelling Insights

The demand framework reflects several important business and technical observations. First, price is the dominant driver of demand in the mobile-phone setting, but it is not the only driver. Product lifecycle, seasonal timing, and festival effects also materially affect purchase volume. Second, different product tiers exhibit different elasticity behavior, so a single static pricing rule would be too crude for the platform. Third, demand should be interpreted not only as a prediction target but as an input to a constrained profit maximization problem. These insights justify the platform's design philosophy, which places optimization and operational decision-making on top of the forecasting layer.

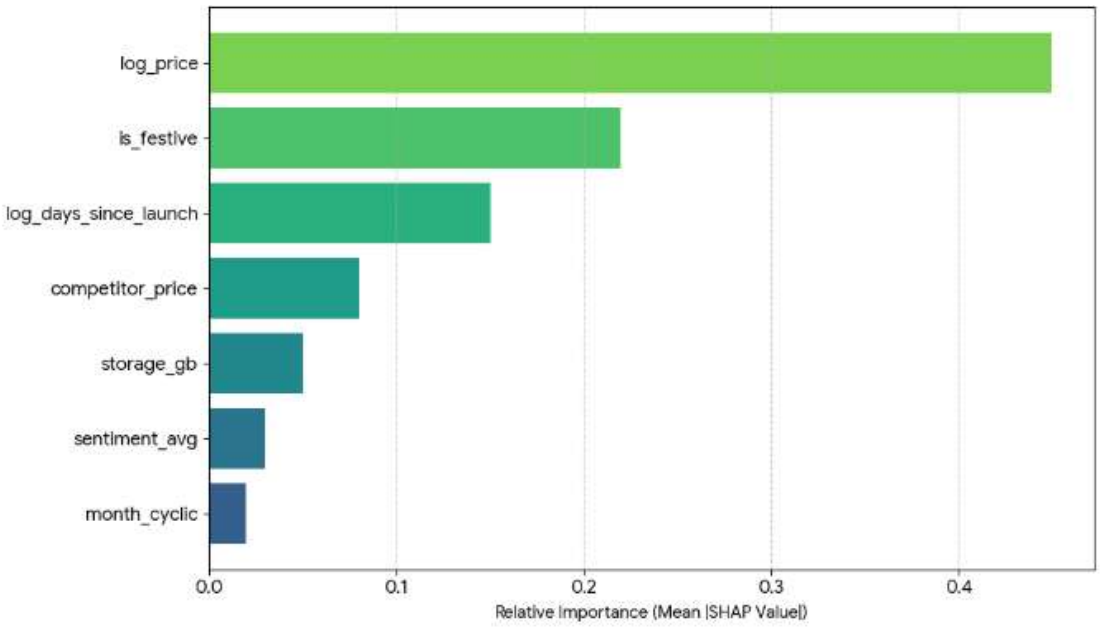

Figure 3.8: *Feature-importance plot*

Chapter 4

Optimization Engine and Profitability Orchestration

4.1 Introduction to Pricing Optimization

Pricing in e-commerce is not simply a matter of selecting the highest price the market may tolerate. In a realistic retail environment, the pricing decision interacts with demand response, inventory risk, shipping and handling costs, competitor positioning, and seasonal demand variation. For this thesis, the platform developed in this thesis treats pricing as a constrained optimization problem rather than a standalone forecasting exercise. The objective is to choose prices and operational settings that maximize **net profit**, not merely revenue or gross margin, across SKUs, time periods, and marketplace contexts. This is the main technical contribution of MTP-II and the part that differentiates it from the earlier MTP-I engine.

The optimization layer takes the normalized demand output from Pipeline 1 and transforms it into a profit landscape over candidate prices. Unlike a static markup method, the engine evaluates how demand changes with price, how service level affects safety stock, and how order grouping influences transport cost. The result is a decision-support system that can recommend prices, service levels, reorder points, and feasible operating bands for each SKU and marketplace setting.

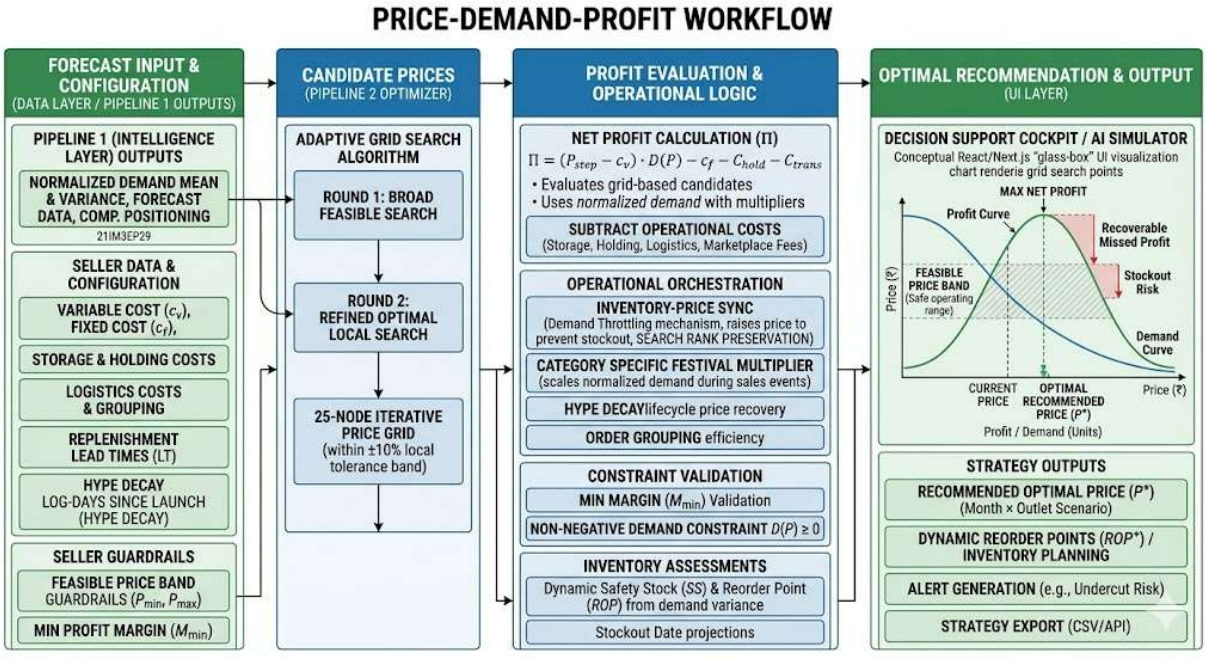

Figure 4.1: *Price-Demand-Profit workflow showing forecast input, candidate prices, profit evaluation, optimal recommendation*

4.2 Problem Formulation

Let $s \in S$ denote a SKU, $m \in M$ denote a time period such as month, and $o \in O$ denote a marketplace or outlet. The platform computes a decision for each SKU-time-marketplace combination. The decision variables include the recommended price $p_{s,m,o}$ and the service level $SL_{s,m,o}$. These are chosen to maximize total net profit over the planning horizon.

The forecasted demand is denoted by $\widehat{D}_{s,m,o}(p)$, where demand depends on the chosen price and on contextual factors supplied by Pipeline 1 and the platform's own event and market logic. In the current implementation, the forecasted base demand is normalized first and then adjusted by the relevant festival multiplier and category-aware logic inside the optimization layer. This prevents double counting of seasonal effects and keeps the optimization formulation clean.

4.2.1 Decision Variables

The main decision variables in the current system are:

$$p_{s,m,o} \in [p_{s,m,o}^{\min}, p_{s,m,o}^{\max}]$$

$$SL_{s,m,o} \in [SL^{\min}, SL^{\max}]$$

where $p_{s,m,o}$ is the recommended price for SKU s in month m and marketplace o, and $SL_{s,m,o}$ is the target service level used to compute safety stock and reorder point. The service level is not treated as a fixed constant in this thesis; it is a decision variable because the best profit outcome often depends on balancing stockout risk against holding cost. This is a major improvement over a fixed-margin or fixed-safety-stock policy.

If needed for implementation detail, the optimizer can also be interpreted as choosing an order batch or replenishment timing implicitly through the reorder-point logic, although the primary explicit decision variable remains price and service level. Order grouping is then handled as an operational consequence of those decisions.

4.2.2 Objective Function

The platform maximizes expected **net profit** over the complete portfolio and planning horizon. A suitable representation is:

$$\max \sum_{s \in S} \sum_{m \in M} \sum_{o \in O} [\, p_{s,m,o}\, \widehat{D}_{s,m,o}(p_{s,m,o}) - c_{s,m,o}^{\text{var}}\, \widehat{D}_{s,m,o}(p_{s,m,o}) - C_{s,m,o}^{\text{hold}} - C_{s,m,o}^{\text{log}} - C_{s,m,o}^{\text{stockout}} - C_{s,m,o}^{\text{misc}}]$$

Here:

- $p_{s,m,o}\, \widehat{D}_{s,m,o}(p_{s,m,o})$ is the expected revenue,

- $c_{s,m,o}^{\text{var}}$ is the unit variable cost,
- $C_{s,m,o}^{\text{hold}}$ is the expected holding or storage cost,
- $C_{s,m,o}^{\text{log}}$ is the transport or logistics cost,
- $C_{s,m,o}^{\text{stockout}}$ is the penalty associated with lost sales or stockout risk,
- $C_{s,m,o}^{\text{misc}}$ captures any additional operational cost terms that are relevant in implementation, such as marketplace fee effects when included in the net profit calculation.

This objective is the correct one for a profitability platform because it forces the optimizer to account for operational frictions rather than chasing top-line revenue alone. The system is therefore capable of preferring a price that sells slightly fewer units if that price reduces stockout incidence, logistics burden, or unnecessary inventory accumulation and thereby improves total net profit.

4.2.3 Constraints

The optimization is subject to the following core constraints:

$$p_{s,m,o}^{\min} \leq p_{s,m,o} \leq p_{s,m,o}^{\max}$$

$$\widehat{D}_{s,m,o}(p_{s,m,o}) \geq 0$$

$$\text{Margin}_{s,m,o}(p_{s,m,o}) \geq \text{Margin}_{s,m,o}^{\min}$$

$$0 \leq SL_{s,m,o} \leq 1$$

$$ROP_{s,m,o} \leq I_{s,m,o}^{\text{available}} + \text{incoming}_{s,m,o}$$

where $I_{s,m,o}^{\text{available}}$ is the currently available inventory and $\text{incoming}_{s,m,o}$ represents incoming replenishment if applicable. The feasible price range ensures that the recommendation remains realistic and respects market or business guardrails. The margin constraint ensures that the system does not recommend prices that are technically optimal from a demand perspective but operationally unacceptable from a cost perspective. The inventory feasibility condition prevents the optimizer from assuming sales that cannot be fulfilled.

4.3 Demand-Adjusted Profit Evaluation

For each candidate price, the optimization engine evaluates demand through the forecasting layer and then computes the corresponding profit. The key idea is simple: rather than approximating the relationship with a static rule, the engine explicitly computes the profit response over a candidate price range and selects the price that yields the highest expected net profit. This is consistent with the earlier MTP-I logic, but MTP-II extends it by incorporating inventory and logistics effects in the profit function.

In the current system, the demand fed into this computation is the normalized base demand from Pipeline 1, adjusted by the category-specific festival multiplier when relevant. If the SKU is in a highly elastic category, the optimizer may also expand the candidate price band to search a wider region. This is important because a narrow fixed band can miss the true profit peak for certain SKUs, especially during sales periods or for highly price-sensitive products. The adaptive expansion of the band is therefore a practical improvement over a fixed search window.

4.3.1 Price Search Logic

The implementation can be described as an adaptive grid search over candidate prices. For each SKU, month, and marketplace, the optimizer creates a price grid around the reference price. The standard search range is centered on the current or baseline price, and the range can expand for highly elastic SKUs. Each candidate price is evaluated using the profit function, and the best candidate is retained.

Formally, if $P_{s,m,o}$ is the set of candidate prices, then:

$$p^*_{s,m,o} = \arg \max_{p \in P_{s,m,o}} \Pi_{s,m,o}(p, SL_{s,m,o})$$

where $\Pi_{s,m,o}$ denotes expected net profit for that price and service-level combination. The system can be implemented either as a joint discrete search over (p, SL) or as a nested search in which service levels are evaluated first and the best price is chosen inside each service-level scenario. Both views are mathematically equivalent if the same candidate sets are used.

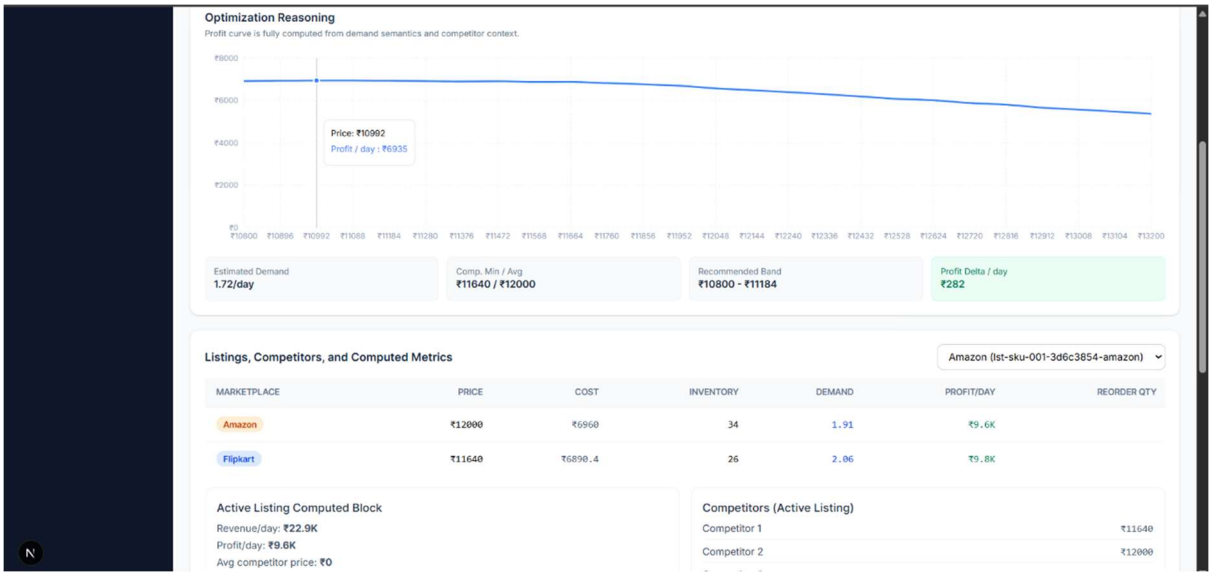

Figure 4.3.1: *Price Optimization – Profit Maximization*

4.3.2 Elasticity-Aware Band Expansion

The platform does not treat all SKUs identically. For highly elastic SKUs, the demand curve is steeper and the profit maximum may lie outside a narrow ±10% band. In such cases, the optimizer expands the search interval so that it can capture the true profit peak rather than stopping at an artificially constrained price neighborhood. This feature is important because different mobile-phone tiers and product families can show very different price sensitivity. Budget and mid-range SKUs tend to react

more strongly to price changes than premium products, which makes this adaptive logic especially useful.

4.4 Inventory-Aware Profitability and Service Level Optimization

A major extension in MTP-II is the explicit treatment of inventory and service level. In MTP-I, the pricing objective primarily focused on demand and profit. In the current platform, service level is part of the optimization decision itself because inventory risk affects realized net profit. A price that sells too aggressively may cause stockouts, lost sales, and reranking or recovery costs; a price that is too conservative may inflate holding costs and reduce turnover. The optimal outcome lies in balancing these effects.

The service level determines the safety stock buffer. Using the forecasted demand mean and variance, the system computes safety stock as a function of service level. A standard form is:

$$SS_{s,m,o} = z(SL_{s,m,o}) \cdot \sigma_{d,s,m,o} \cdot \sqrt{L_{s,o}}$$

where $z(SL_{s,m,o})$is the service-level quantile, $\sigma_{d,s,m,o}$is the standard deviation of demand for the SKU, and $L_{s,o}$is the replenishment lead time. Then the reorder point is:

$$ROP_{s,m,o} = \mu_{d,s,m,o} \cdot L_{s,o} + SS_{s,m,o}$$

where $\mu_{d,s,m,o}$is mean demand over the lead-time window. This formulation ensures that the reorder point is not arbitrary: it is anchored in both predicted demand and a chosen service-level target. Higher service levels increase safety stock, reduce stockout probability, and also increase holding cost, which is exactly why service level should be optimized rather than hard-coded.

The implication is that the platform can recommend different service levels for different SKUs, marketplaces, or time periods. For example, a highly elastic SKU with a high risk of demand spikes may justify a higher service level, while a slow-moving SKU may not. The optimizer therefore becomes a profitability orchestrator rather than a narrow price calculator.

4.4.1 Order Grouping and Logistics Efficiency

The platform also incorporates order grouping as an operational cost-reduction mechanism. Instead of replenishing each SKU independently whenever it touches its reorder point, the system can group nearby replenishment events when the timing and volume make that efficient. The rationale is that consolidated ordering can lower transport cost even if it slightly increases holding cost. Since net profit is the objective, the engine can accept a small holding-cost increase if the freight savings are larger overall. This is particularly relevant in Indian e-commerce, where logistics and fulfillment costs can materially affect margins.

4.5 Festival-Aware Optimization

The platform explicitly separates normalized demand from festival uplift. Pipeline 1 produces a base demand estimate that is free from historical festival distortion, and Pipeline 2 applies a category-specific festival multiplier to model the current seasonal environment. This is important because the same SKU may behave differently during a major sale event compared to a normal month. The optimizer should therefore not rely on a raw historical series alone. Instead, it should use the base demand estimate plus a controlled multiplier that reflects the current season.

This design is especially valuable for fast-moving mobile-phone campaigns such as festive sale windows. During such periods, the optimizer may justify a lower price if the demand surge is strong enough to improve total net profit. After the event, the system can support price recovery by gradually raising price as elasticity normalizes. This allows the platform to protect margin after the festival rather than remaining stuck at promotional levels longer than necessary.

4.6 Competitor-Aware Guardrails

The current implementation also includes competitor intelligence, which helps the system situate the recommended price in a realistic market context. The platform can compare its own price against stored competitor data and flag undercut risk or unusually high pricing. In production, this logic should be fed by live competitor data; for the current project implementation, saved historical competitor data is used as the working input. The purpose of this module is not to force the optimizer to match the lowest competitor price, but to prevent recommendations that are operationally or strategically implausible in the current market.

4.7 Algorithmic Summary

The optimization logic used in the platform can be summarized as follows:

1. Receive the normalized base demand output from Pipeline 1 for a SKU, time period, and marketplace.
2. Apply the appropriate festival multiplier and contextual adjustments.
3. Construct a candidate price set around the reference price, with adaptive expansion for highly elastic SKUs.
4. For each candidate price, compute expected demand, revenue, cost, holding cost, logistics cost, and stockout risk.
5. For each candidate service level, compute safety stock and reorder point from demand variance and lead time.
6. Evaluate net profit for all feasible (p, SL)combinations.
7. Select the combination that maximizes total net profit while satisfying margin and inventory constraints.
8. Generate explanation outputs: optimal price, feasible band, expected units, service level, safety stock, reorder point, and profit impact.

4.8 Output Structure of the Optimization Engine

The optimization engine returns a structured recommendation package rather than a single price. This is a key product feature because sellers need not only a decision but also the rationale and operational consequences of that decision. The output package can include the following items:

- optimal price,
- feasible price interval,
- expected demand,
- expected net profit,
- implied margin,
- selected service level,
- safety stock,
- reorder point,
- expected stockout risk,
- logistics implication,
- competitor position summary,
- festival sensitivity summary,
- and any alert flags such as low stock or undercut risk.

These outputs are consumed by the dashboard layer, where they are displayed through recommendation cards, profit curves, sensitivity visuals, and scenario comparison panels. This is the main reason the system functions as a product rather than just a model.

Chapter 5

Integrated Platform Modules and Product Workflow

5.1 Introduction

Our system is not limited to pricing optimization alone. It is implemented as a broader AI-powered pricing platform that combines forecasting, optimization, inventory reasoning, competitor intelligence, event sensitivity, and user-facing analytics into a unified workflow. This chapter describes the integrated modules that sit around the core optimization engine and make the system usable as a product. In the current implementation, these modules are already part of the working platform and are not merely conceptual future additions.

The importance of this integration is that pricing recommendations are only one part of a seller's actual decision environment. A seller must also think about stock availability, replenishment timing, competitor behavior, festival demand surges, logistics cost, and the practical consequences of changing price too often or too aggressively. The platform therefore extends the optimization layer into a decision-support and profitability-orchestration layer.

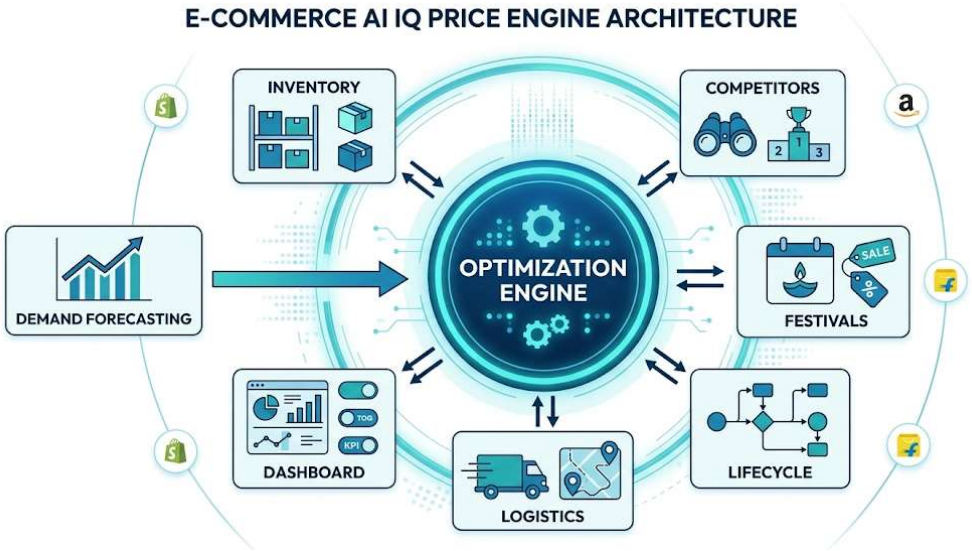

Figure 5.1: *Platform modules*

5.2 Inventory-Price Synchronization

One of the most important extensions in MTP-II is the explicit coupling of price with inventory dynamics. In practical e-commerce settings, price directly influences the rate at which inventory is depleted, and inventory availability in turn affects realized profit. If a seller underprices a fast-moving SKU, stock may run out too early, causing lost sales and hidden recovery costs. If the seller overprices a slow-moving SKU, holding cost and storage burden can accumulate unnecessarily. The platform

therefore uses inventory-aware pricing so that price decisions are consistent with replenishment constraints and service-level targets.

In the current system, inventory is not treated as a passive record. It is an active control variable that affects recommended pricing, safety stock, reorder timing, and shipment grouping. The system estimates demand variability from the forecast layer, converts that into safety stock through the chosen service level, and then uses reorder-point logic to determine when replenishment should be triggered. This is what allows the platform to protect net profit rather than simply maximize short-term unit sales.

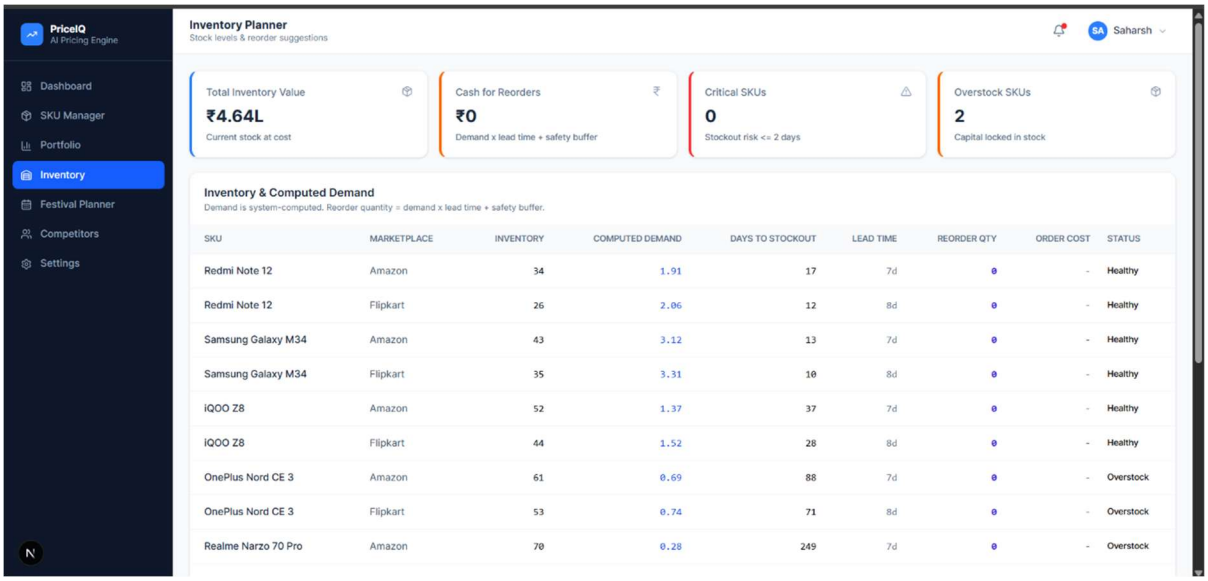

Figure 5.2: *Inventory Planner Screenshot*

5.3 Service Level as an Optimization Variable

A major improvement in MTP-II is that service level is not fixed externally. Instead, it is treated as a decision variable inside the optimization process. This allows the engine to choose between a higher service level with greater holding cost and a lower service level with greater lost-sales risk. In real retail operations, the profit-optimal choice is rarely one extreme or the other; it is the balance point where total net profit is highest.

The system uses forecasted demand variance, together with lead time, to compute safety stock for each SKU-marketplace-time combination. A higher service level implies a higher z-value, which increases buffer stock and reduces the probability of a stockout. This is especially important for mobile phones or other fast-moving SKUs where lost availability can also affect search ranking and future conversion. The service level is optimized for profit, not for the abstract goal of “avoiding stockouts at all costs.”

5.4 Competitor Intelligence Module

The platform also includes a competitor intelligence layer. This module is used to compare the seller’s current price against competitor prices, identify undercut risk, and contextualize the recommendation within a realistic market environment. In the current project implementation,

historical or previously saved competitor data is used; in production, this logic should be driven by live market data so that recommendations remain current and market-aware.

The role of competitor intelligence is not to force a simple lowest-price reaction. Instead, it acts as a guardrail and an interpretive layer. A recommendation that is mathematically optimal may still be strategically undesirable if it positions the seller too far above or below the market range. The module therefore helps the platform stay commercially realistic while preserving the main objective of net profit maximization.

The competitor module also supports a user-facing analytics workflow. Sellers can inspect the historical spread between their price and competitor prices, understand undercut frequency, and identify products where price defense is necessary. This is useful both for product positioning and for explaining why the optimizer chose a particular output.

5.5 Festival and Event-Sensitive Pricing

Another key module is the festival-aware pricing layer. Indian e-commerce demand is strongly influenced by festivals and sale events, and mobile-phone demand in particular can vary sharply during event windows. To avoid contaminating the base demand model with repeated seasonal spikes, the system uses normalized demand from Pipeline 1 and then applies category-specific festival multipliers inside Pipeline 2. This creates a cleaner separation between long-run demand estimation and short-run event adjustment.

The festival module allows the optimizer to adjust prices and inventory decisions for periods such as Diwali, Big Billion Days, or other category-relevant sale events. The logic is not restricted to a fixed uplift; instead, it can be statistically derived for the category under consideration. In the mobile-phone proof-of-concept, this means that the system can exploit a temporary surge in demand while still protecting post-event margin through price-recovery logic.

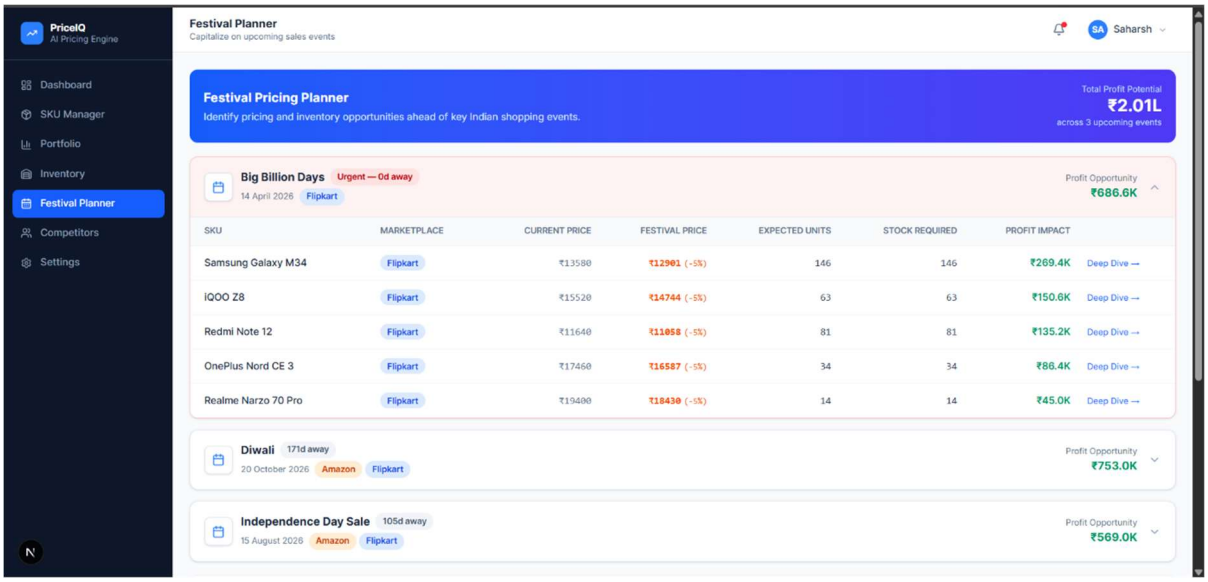

Figure 5.5: *Festival Planner Screenshot*

5.6 Lifecycle Pricing and Price Recovery

The platform also models product lifecycle effects, which are especially relevant for mobile phones. As a product ages, demand tends to decay, and the optimal price should adjust accordingly. The system captures this through a hype-decay or launch-age variable. Rather than waiting for a sharp sales fall and then applying a large markdown, the platform can support gradual price adjustment and price recovery after promotional periods. This is a more realistic and more profitable strategy than manual step-down pricing.

Price recovery is useful after festivals or temporary discounting periods. Once the event-driven elasticity effect begins to decay, the system can gradually raise price back toward the baseline optimum. This avoids margin bleed caused by leaving promotional prices active for too long.

5.7 Order Grouping and Logistics Efficiency

The platform includes a logistics-aware order grouping mechanism. When multiple SKUs are approaching replenishment thresholds around similar times, the system can consolidate ordering decisions instead of treating each SKU independently. This reduces the number of shipments and can lower transport cost, even if it slightly increases holding cost. Since the objective is net profit, the optimizer can choose this trade-off when the savings from shipment consolidation exceed the additional inventory cost.

This is a significant practical feature because logistics cost is often ignored in price-only systems. In Indian e-commerce, freight, packaging, and fulfillment overhead can materially affect the true profitability of a SKU. Order grouping therefore turns the platform into a more complete operational decision engine. It is not just telling the seller what to charge, but helping the seller decide how to replenish efficiently and profitably.

5.8 User Workflow and Decision Support

The product is designed around an intuitive workflow. The seller enters or uploads product and operational data, the platform computes demand-adjusted recommendations, and the dashboard presents the output in a form that can be inspected, simulated, and applied. This is important because the system is intended to be a usable product, not only a research artifact. The user should be able to see the suggested price, the feasible band, the expected profit impact, and the inventory implication in one place.

The dashboard supports recommended actions, alert panels, and drill-down views for SKUs, competitors, festivals, inventory, and portfolio-level analytics. The platform also includes scenario tools for simulating price changes before applying them. This helps sellers understand the consequences of recommendations and builds trust in the system. A clear product narrative should be maintained here: the platform turns complex optimization into an actionable operational workflow.

Chapter 6

System Implementation and Deployment Architecture

6.1 Introduction

This chapter describes how the platform is implemented at the software and system level. The implementation follows a modular full-stack architecture with a Next.js frontend, a FastAPI backend, and MongoDB as the persistence layer. This design is suitable for an AI-powered product because it separates user interaction, business logic, and data management cleanly while still allowing the platform to remain responsive and extensible.

The implementation is intentionally product-oriented. The frontend is designed as a dashboard and decision-support interface; the backend provides services for pricing, demand, competitor, inventory, and festival logic; and the database stores multi-tenant seller-specific data. The architecture is structured to support current evaluation on the mobile-phone proof of concept while remaining generic enough for future category expansion.

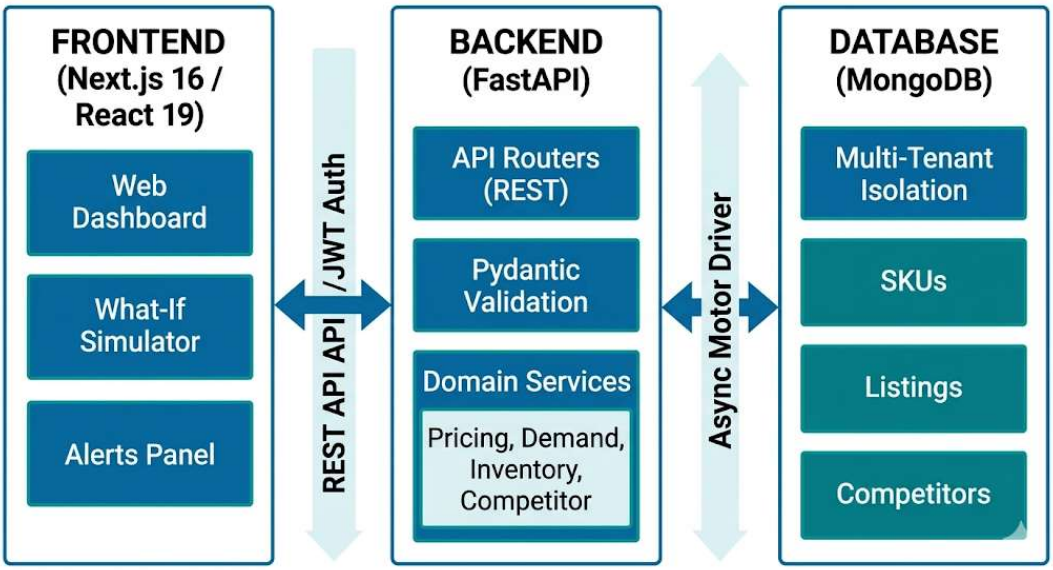

***Figure 6.1**: Backend-Frontend-Database Architecture diagram*

6.2 Frontend Architecture

The frontend is implemented using React 19 with Next.js 16.1 in the App Router architecture. Tailwind CSS v4 is used for styling, and Recharts is used for visualization. Lucide React provides iconography, and the interface is built around analytical cards, charts, data tables, and simulation controls. This makes the platform suitable for a seller-facing workflow where decisions need to be inspected visually rather than through raw model output alone.

The frontend includes dashboard views for overall metrics, recommended actions, alerts, SKUs, competitor intelligence, festival planning, inventory, and portfolio analytics. Each screen is designed to expose a different layer of the system's reasoning. For example, the dashboard shows top-line KPIs and priority alerts, while the SKU detail view exposes price simulation and profit curves. This structure is important because it bridges the gap between the technical engine and the seller's decision process.

The UI also includes a what-if simulator. Users can adjust price or related parameters and immediately see how the demand curve, profit curve, and stockout estimate change. This simulator is a central product feature because it makes the optimization logic transparent and interactive. The thesis should show at least one screenshot of this simulator and one screenshot of the main dashboard.

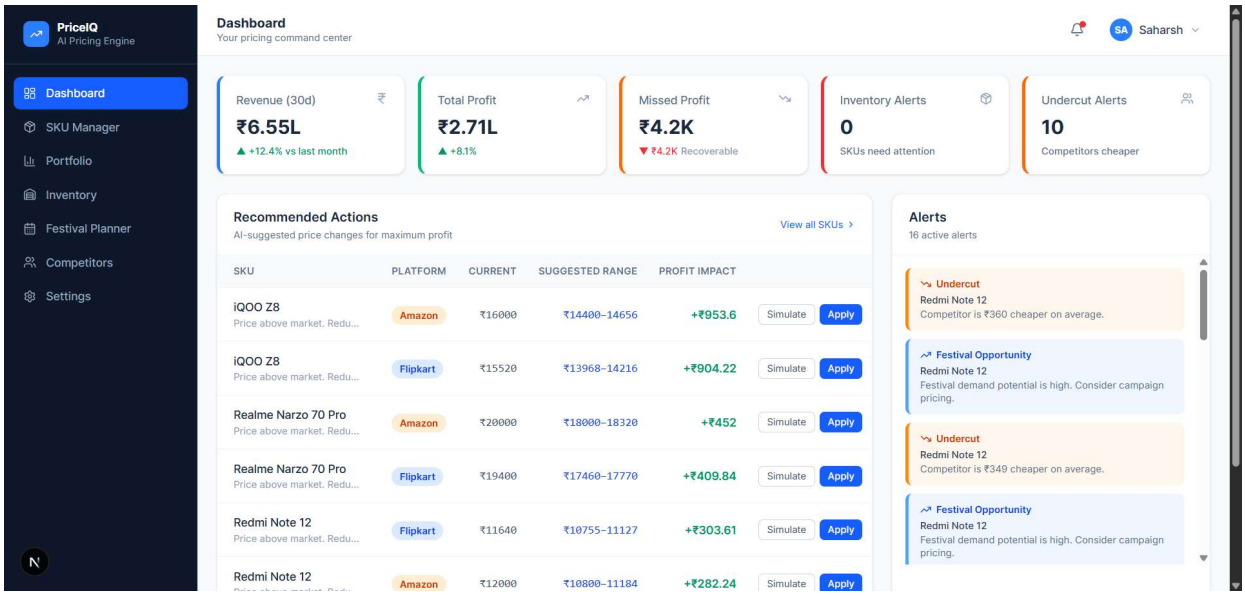

Figure 6.2.1: *Dashboard screenshot*

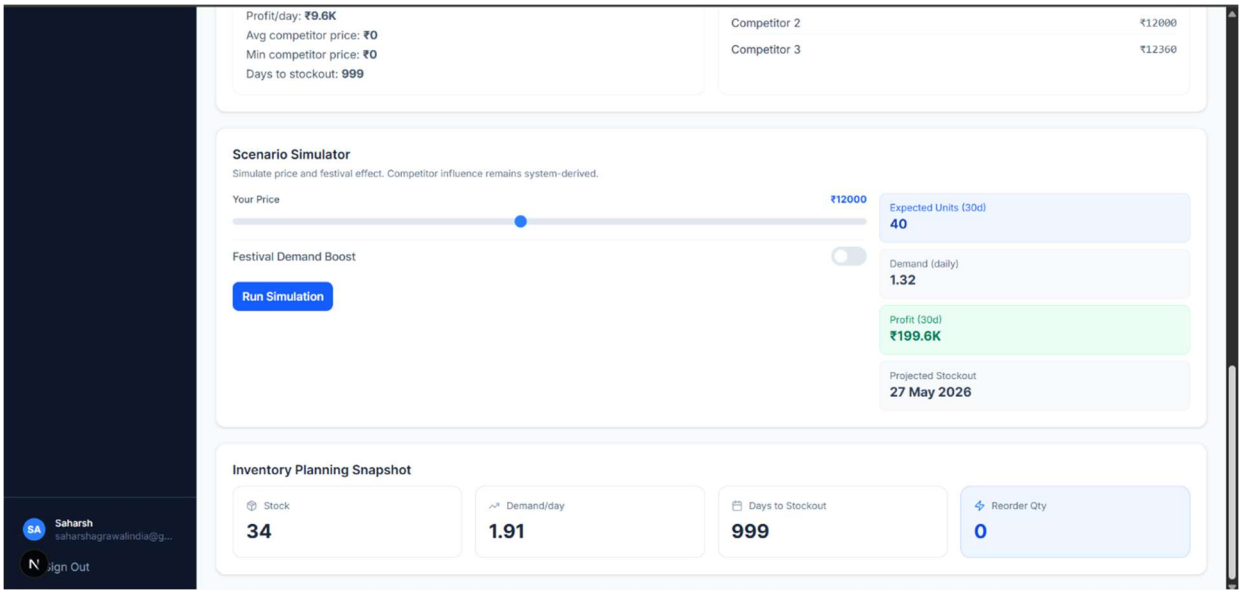

Figure 6.2.2: *What-if simulator screenshot*

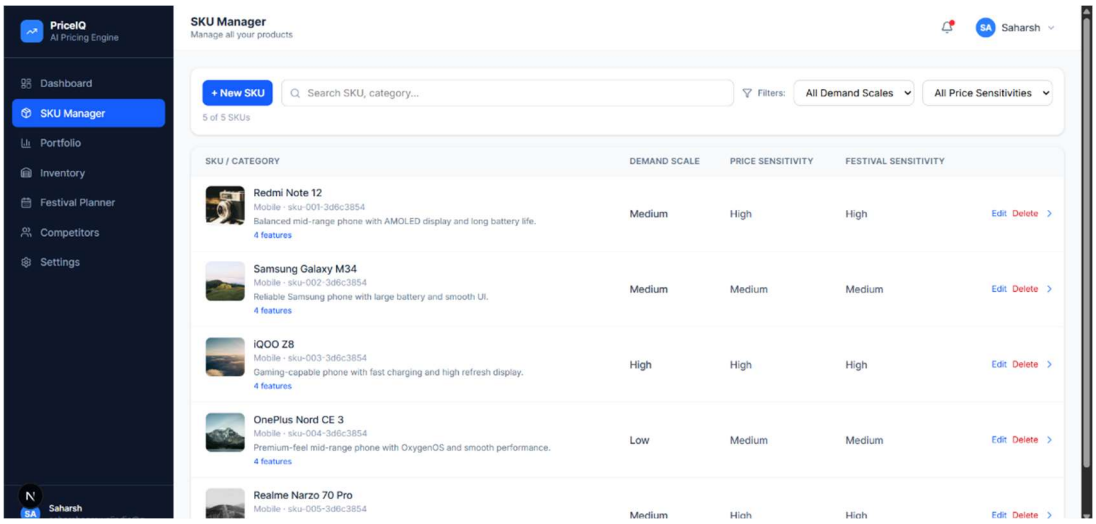

Figure 6.2.3: *SKU Manager screenshot*

6.3 Backend Architecture

The backend is implemented using FastAPI 0.115, which is appropriate for a high-performance service-oriented architecture. The backend exposes REST endpoints for authentication, SKU management, listings, pricing simulation, competitor analysis, festival logic, and inventory-related operations. Pydantic is used for schema validation, while bcrypt and JWT are used for secure authentication.

The backend is organized into domain services such as pricing service, demand service, inventory service, competitor service, dashboard service, and catalog service. This separation ensures that mathematical logic is isolated from route handling and data formatting. The pricing service computes profit curves and recommendations, while the inventory service computes reorder points and stockout-related outputs. The competitor service evaluates undercut risk and historical price spread, and the dashboard service aggregates outputs into a seller-readable summary.

The service layer is especially important because it keeps the pricing math deterministic and testable. The route layer handles requests, authentication, and payload formatting, but the mathematical logic remains inside service functions. This is a good design choice for a thesis project because it makes the implementation easier to reason about, debug, and extend.

6.4 Database Design

The platform uses MongoDB with asynchronous access through the Motor driver. The data model is organized around a multi-tenant structure in which each organization has its own isolated data context. This is important for a SaaS-like product because sellers must not see each other's catalogs or analytics. Authentication tokens therefore carry organizational context, and database queries are filtered by org-specific identifiers.

The core data entities include SKU profiles, listings, competitors, and festival configurations. The SKU profile stores the product identity and core pricing-related attributes. Listings tie an SKU to a marketplace such as Amazon or Flipkart, and they also store price, inventory, lead time, and storage cost. Competitor entries are linked to listings and store competitor price and other market signals.

Festival records store event metadata and uplift assumptions. This schema is sufficient to support the current platform behavior and can be extended later if more granular operational fields are required.

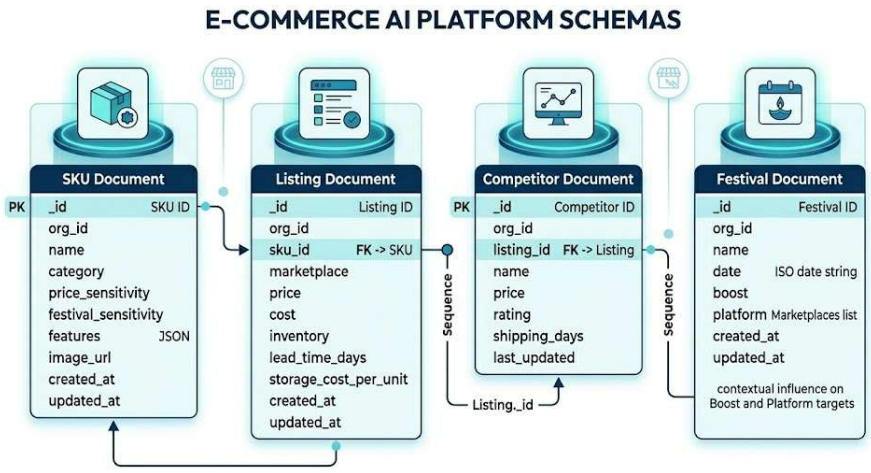

Figure 6.4: *Schema relationship diagram*

6.5 Authentication and Multi-Tenancy

The platform is implemented as a B2B SaaS-style multi-tenant system. Users authenticate through a JWT-based login flow, and the token stores both the user and organization context. This ensures that every query and every action is scoped to the correct seller account. Passwords are protected using bcrypt, and the frontend attaches the token to API requests so that unauthorized access is blocked.

This design is important not only for security but also for product realism. A pricing platform intended for business use must support account-level isolation, role-based control, and secure access to proprietary pricing and inventory data. The thesis should state this briefly, because it shows that the platform is designed with deployment in mind rather than as a one-off academic prototype.

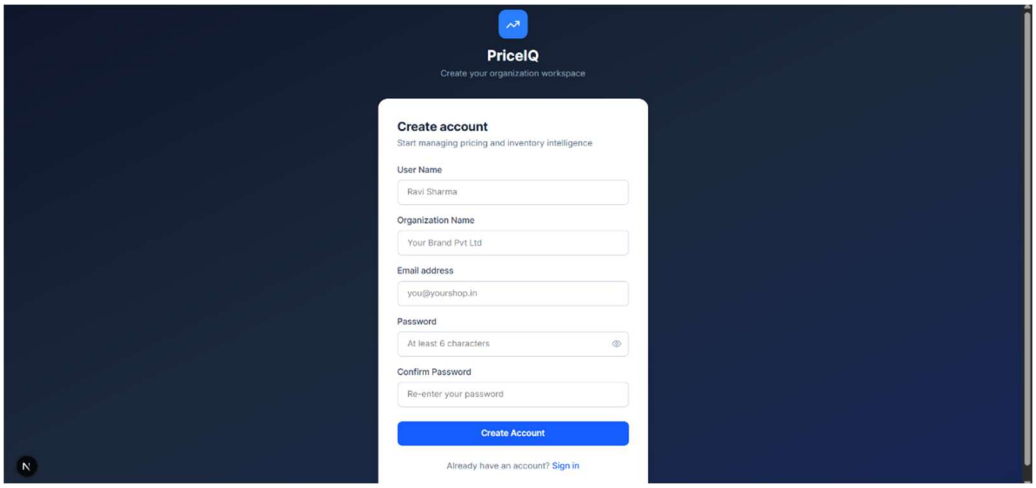

Figure 6.5: *Sign-up page screenshot*

6.6 API and Route Structure

The API is organized into modular route groups. Authentication routes handle registration and login. SKU and listing routes support catalog management. Pricing routes expose optimization and simulation logic. Festival routes manage event-based signals, while competitor and inventory routes expose market and stock-related analytics. This route structure keeps the implementation maintainable and aligns with the domain structure of the product.

6.7 Data Flow Across the Stack

A complete user request flows through the stack as follows. The frontend sends a request from the dashboard or simulator. The backend authenticates the request and fetches the relevant seller context from MongoDB. The pricing or inventory service computes the mathematical output using the forecast input and business constraints. The backend then formats the response and sends it back to the frontend, where it is visualized as a recommendation, curve, or alert. This architecture provides a clear separation between storage, computation, and presentation.

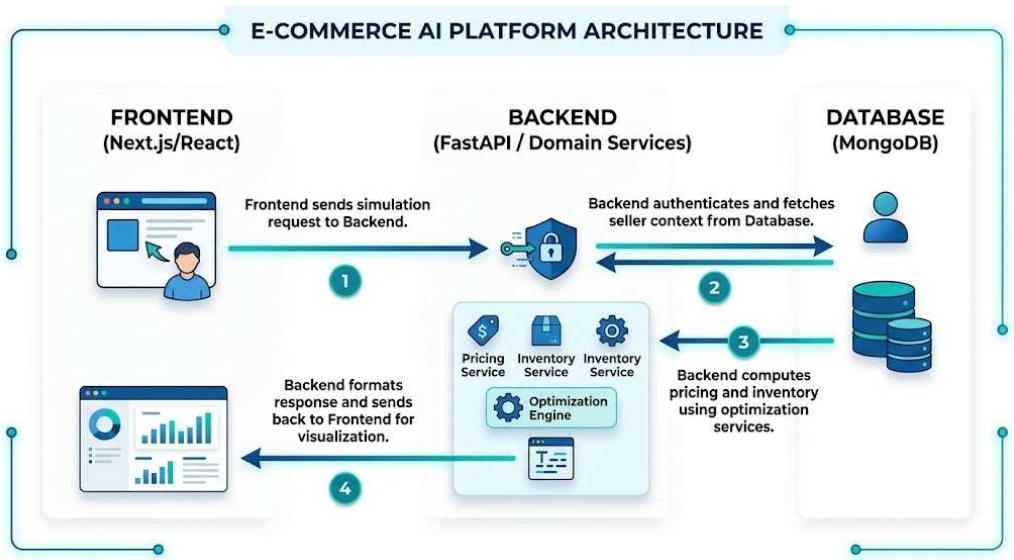

Figure 6.7: *End-to-end request-response flow*

6.8 Deployment Considerations

Although the focus is on the current implementation, the system design is already suitable for deployment as a product prototype. The frontend and backend are modular enough to be hosted separately, the database layer is structured for multi-tenant use, and the product workflow naturally supports API-based onboarding. This means the platform can be adapted for future retailer pilots where live data is collected over time and the forecasting-optimization cycle becomes more reliable.

The current project also reflects a realistic cold-start strategy. When a new retailer or category is onboarded, the platform can begin by collecting live transactions and market data before generating full-confidence recommendations. This makes the system suitable for practical adoption rather than only for retrospective evaluation.

Chapter 7

Experimental Results and Impact Analysis

7.1 Introduction

This chapter evaluates the performance and practical impact of the proposed AI-powered pricing and profitability optimization platform. While earlier chapters established the mathematical formulation and system design, the objective here is to demonstrate that the system produces **measurable improvements in real-world business metrics**, particularly **net profit**, which is the primary optimization objective of the platform.

Due to the limited availability of large-scale proprietary retail datasets, the evaluation is conducted as a **controlled backtesting simulation** using a representative subset of the **mobile phone category**. This category is chosen because of its high price sensitivity, rapid product lifecycle, strong festival-driven demand variation, and availability of sufficient public data to construct a meaningful proof-of-concept. However, the system itself remains **category-agnostic**, and the results are intended to demonstrate general applicability rather than category-specific tuning.

The results are structured across multiple controlled experiments, each isolating a key dimension of the system:

- Pricing-only optimization vs full-system optimization
- Festival-aware pricing
- Demand model validation
- Service-level optimization
- Real-world operational complexity scenarios

7.2 Experimental Setup

7.2.1 Dataset and Scope

The experiments are conducted on a curated dataset representing a small portfolio of **mobile phone SKUs**. The dataset integrates:

- historical sales signals,
- product attributes,
- inferred competitor pricing,
- and event-based demand patterns.

Demand forecasts are obtained from Pipeline 1 and treated as a **black-box input**, as described in Chapter 2. These forecasts are normalized to remove historical seasonal bias and are later adjusted using festival multipliers within the optimization engine.

7.2.2 Simulation Parameters

The primary backtesting scenario uses the following setup:

- Number of SKUs: 5 (mid-range mobile phones)
- Average unit cost: ₹12,000
- Time horizon: 12 months
- Storage cost: ₹8 per unit per day
- Logistics cost: ₹150 per order
- Baseline service level: 95%

These values are chosen to reflect realistic mid-market Indian e-commerce operations while keeping the simulation interpretable.

7.3 Comparative System Evaluation (Three-Tier Analysis)

To isolate the contribution of each system component, the platform is evaluated across three progressively advanced configurations:

- **Tier 1:** Static pricing baseline
- **Tier 2:** MTP-I (demand-aware pricing only)
- **Tier 3:** MTP-II (full system: pricing + inventory + logistics + service level)

7.3.1 Aggregate Performance Comparison

Metric	Tier 1: Static	Tier 2: MTP-I	Tier 3: MTP-II
Units Sold	10,000	11,400	10,850
Revenue	₹18.00 Cr	₹19.95 Cr	₹19.42 Cr
Gross Margin	15.0%	14.2%	16.5%
Holding + Logistics Cost	₹45.0 L	₹58.5 L	₹38.0 L
Stockout Rate	8.0%	14.5%	1.2%
Net Profit	₹2.25 Cr	₹2.25 Cr	₹2.82 Cr
Net Profit Uplift	Base	0%	**+25.3%**

Table 7.3.1: *Annual Financial & Operational Performance*

7.3.2 Key Observations

The results highlight a critical insight:

- MTP-I increases **revenue**, but not **net profit**
- This is because it ignores inventory and logistics constraints

- Higher demand leads to:
    - more stockouts
    - higher logistics fragmentation
    - increased holding inefficiencies

MTP-II resolves this by introducing **Inventory-Price Synchronization**, which slightly reduces total volume but significantly improves:

- cost efficiency
- fulfillment stability
- margin quality

This leads to a **25.3% increase in net profit**, which validates the core thesis objective.

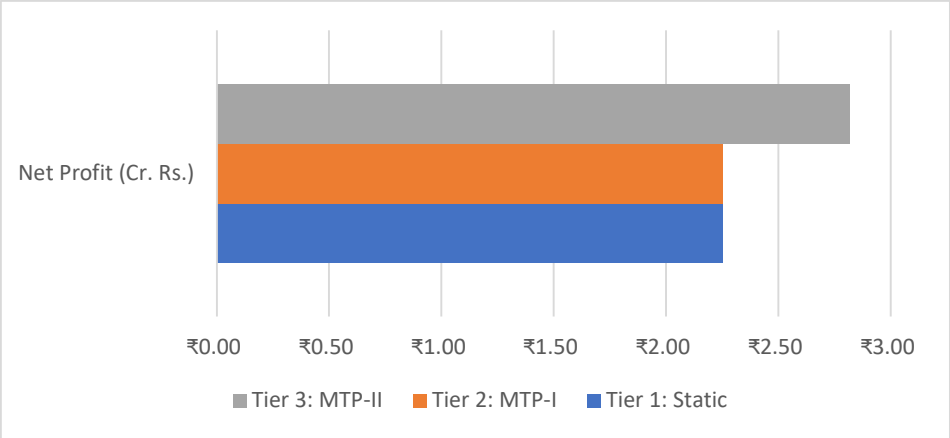

Figure 7.3.2: *Bar chart comparing Net Profit across tiers*

7.4 Festival-Aware Pricing and Price Recovery

7.4.1 Experimental Setup

A high-demand festival scenario is simulated over a **30-day window**, including a 5-day peak event (e.g., Diwali / Big Billion Days).

Pipeline 1 provides **normalized demand**, and Pipeline 2 applies a **category-specific festival multiplier (1.45×)**.

7.4.2 Results

Metric	Without Festival Logic	With Festival Logic	Impact
Optimal Price	₹14,500	₹13,800	Engine dynamically expands 10% band
Units Sold (Peak)	450	820	Captured heightened elasticity

Peak Profit	₹11.25 L	₹14.76 L	**+ 31.2% Profit Surge**
Margin Recovery Time	14 days	6 days	Automated elasticity decay tracking

Table 7.4.2: *Festival Performance*

7.4.3 Interpretation

The results demonstrate that:

- The system correctly identifies increased price elasticity during events
- It expands the price search band and lowers price strategically
- It captures significantly higher volume without sacrificing margin

The **price recovery mechanism** ensures that:

- prices are not left artificially low post-event
- margin is restored efficiently

This validates the separation between:

- **base demand modelling (Pipeline 1)**
- **event-aware optimization (Pipeline 2)**

7.5 Demand Model Validation and Elasticity Insights

7.5.1 Forecast Accuracy

Tier	MAPE	Elasticity	Business Implication
Budget	17.2%	-2.85	High volume sensitivity; 1% price drop yields 2.85% demand spike.
Mid-Range	21.8%	-1.92	Sweet spot for dynamic pricing; balanced volume-to-margin ratio.
Flagship	18.5%	-0.85	Premium brand loyalty dampens price sensitivity; engine avoids deep discounts.

Table 7.5.1*: Forecast Accuracy*

7.5.2 Key Insights

- Mid-range SKUs show the best forecast accuracy
- Budget phones are highly elastic → strong pricing leverage
- Flagship phones are inelastic → pricing changes have limited impact

This validates the system's ability to:

- differentiate SKU behavior
- avoid over-discounting premium products
- exploit high elasticity segments

This also justifies the use of **elasticity-aware optimization** in Chapter 4.

7.6 Service Level Optimization

7.6.1 Results

Service Level	Holding Cost	Lost Sales	Net Profit	System Diagnosis
90%	₹1.8 L	₹4.2 L	₹14.5 L	Under-stocked. Lost sales erode profit.
95%	₹2.6 L	₹1.1 L	**₹16.8 L**	**Optimal Balance.**
98%	₹4.1 L	₹0.3 L	₹15.2 L	Over-stocked. Storage costs kill margin.
99.9%	₹7.5 L	₹0.0 L	₹11.4 L	Capital inefficient.

Table 7.6.1: *Service Level Optimization*

7.6.2 Interpretation

- Very high service levels are inefficient due to holding cost
- Very low service levels lose revenue
- The optimal point emerges endogenously (~95%)

This proves that:

- service level must be optimized, not fixed
- the system correctly balances risk vs cost

7.7 Real-World Operational Complexity Scenarios

This section demonstrates the **"human vs AI" gap**, which is crucial for both academic and startup positioning.

7.7.1 Lifecycle Pricing (Hype Decay)

Metric	Manual Approach (Step-Down Pricing)	AI Engine (Continuous Hype Decay)	The "Why" (Lost Profit Reason)
Pricing Strategy	₹18,000 for 90 days, then slashed to ₹15,300.	Smooth weekly decay from ₹18,000 down to ₹16,200.	Humans wait too long to discount, requiring deeper cuts later to clear aging stock.

Average Selling Price	₹16,650	**₹17,100**	AI captures buyers willing to pay ₹17k who were ignored by the manual jump to ₹15k.
Units Cleared	1,200	1,350	AI keeps momentum steady.
Total Gross Profit	₹31.8 L	**₹41.8 L**	**+31.4% Profit.** Manual pricing leaves money on the table in months 4-6.

***Table 7.7.1**: Lifecycle Pricing (Hype Decay)*

Insight: AI captures intermediate willingness-to-pay that humans miss.

7.7.2 Inventory Crisis Management

Metric	Human Seller (Reactive)	AI Engine (Demand Throttling)	The "Why" (Lost Profit Reason)
Action Taken	Keeps price flat at ₹12,000.	Raises price to ₹13,200 (+10%).	Humans rarely monitor daily stock-velocity vs. lead-time ratios.
Stockout Duration	5 Days OOS (Stock hits zero early).	**0 Days OOS** (Inventory perfectly stretched).	Human sold out too fast.
Gross Margin (Low Stock)	₹1.8 L	**₹2.6 L**	AI makes premium margins on the final units.
Post-Restock Penalty	₹45,000 spent on ads to regain rank.	**₹0 spent.** Organic rank preserved.	Stockouts carry a hidden "re-ranking tax" that destroys future profit.

***Table 7.7.2:** Inventory Crisis Management*

Insight: AI throttles demand using price instead of running out of stock.

7.7.3 Logistics Optimization

Metric	Manual (Siloed Reordering)	AI Engine (Order Grouping)	The "Why" (Lost Profit Reason)
Number of Shipments	14 Shipments	**6 Grouped Shipments**	Humans reorder per SKU; AI reorders per portfolio.
Total Freight Costs	₹1.05 L	**₹45,000**	Grouped LTL (Less-than-Truckload) saves massive overhead.
Holding Costs	₹22,000	₹28,000	AI holds slightly more buffer stock to align order dates.
Net Logistics Drain	₹1.27 L	**₹73,000**	**42% Reduction in logistics drain**, flowing directly to Net Profit.

***Table 7.7.3:** Logistics Optimization*

Insight: Order grouping reduces logistics drain significantly.

7.7.4 Festival Discount Strategy

SKU Category	Human Strategy (Flat 15% Off)	AI Engine Strategy (Elasticity-Mapped)	Resulting Profit Difference
Budget Phone	-15% Discount	-18% Discount	AI drops price *more* than the human, driving a 300% volume spike that crushes human profit.
Mid-Range	-15% Discount	-12% Discount	AI saves 3% margin per unit; volume remains identical to the 15% cut.
Flagship	-15% Discount	-5% Discount	AI prevents a massive margin bleed. Flagship buyers don't need a 15% cut to convert.
Portfolio Profit	₹8.5 L	**₹12.2 L**	**+43.5% Profit.** The human "gut feeling" strategy over-discounts premium items and under-discounts budget items.

Table 7.7.4: *Festival Discount Strategy*

Insight: AI avoids blanket discounting → +43.5% portfolio profit.

7.8 Overall System Insights

Across all experiments, the following conclusions emerge:

1. **Revenue ≠ Profit**
   Demand-only optimization is insufficient
2. **Inventory-awareness is critical**
   Stockouts and holding costs dominate profit
3. **Elasticity-driven pricing is essential**
   Different SKUs require different strategies
4. **Festival effects must be modelled separately**
   Historical data alone is insufficient
5. **Operational complexity exceeds human capability**
   The system handles multi-variable optimization at scale

Chapter 8

Starting-up and Monetization

8.1 Business Model & Opportunity

8.1.1 Value proposition

AI-Powered Price Optimisation delivers measurable profit uplift for online sellers by using forecasted demand and outlet-level optimisation to recommend prices that maximize profit while respecting business constraints (min margin, cost). Value drivers:

- Direct: higher gross profit per SKU through better prices.
- Indirect: reduced manual pricing effort, better marketplace competitiveness, and faster what-if scenario analysis.
- Strategic: modular service can expand into multi-SKU, inventory-aware, and marketplace-integrated offerings.

8.1.2 Target customers

Primary: Indian mid-market D2C brands and SMB sellers on marketplaces (platforms acting as "outlets" in our model).
Secondary: marketplace SaaS partners, e-commerce enablers, larger brands (enterprise tier).

8.1.3 Monetization levers

- **SaaS subscription** (monthly/annual) per SKU or per seller account.
- **Usage / compute surcharge** for heavy forecasting or on-demand re-training.
- **Professional services** for integration, pilot runs, and advanced custom modelling.
- **Revenue share / success fee** (optional): small % of incremental profit captured (needs careful legal/accounting design).

8.2 Business Model Canvas

Key Partners

- Forecasting/data partners (future)
- E-commerce marketplaces / aggregator APIs
- Data providers (pricing, competitor feeds — optional)
- Cloud providers (AWS/GCP/Azure)
- Early pilot customers / merchants

Key Activities

- Model development & maintenance (forecast + optimiser)
- Data engineering & pipelines
- Product engineering (UI, APIs)
- Sales & customer success (pilot setup, onboarding)
- Monitoring & analytics

Key Resources

- Core engineering team (ML + backend + frontend)
- Forecasting model (pipeline-1 team / collaborator)
- Cloud infrastructure & storage
- Data (historical sales / pricing)

Value Propositions

- Outlet × Month profit optimisation
- Explainable recommendations & edge signals
- Easy exports & manual-accept workflows
- Tunable parameters for business constraints

Customer Relationships

- Pilot onboarding (hands-on)
- SLA for enterprise customers
- Self-serve + documentation for SMBs

Channels

- Direct sales / inside sales to D2C brands
- Partnerships with marketplace enablers / agencies
- Content marketing (case studies) + conferences

Customer Segments

- SMB D2C sellers (primary)
- Aggregator platforms (secondary)
- Enterprise brands (advanced tier)

Cost Structure

- Fixed: product development, core infra, licensing
- Variable: compute costs, support, data acquisition

Revenue Streams

- Monthly subscriptions (tiered)
- Professional services
- Success fees (optional)

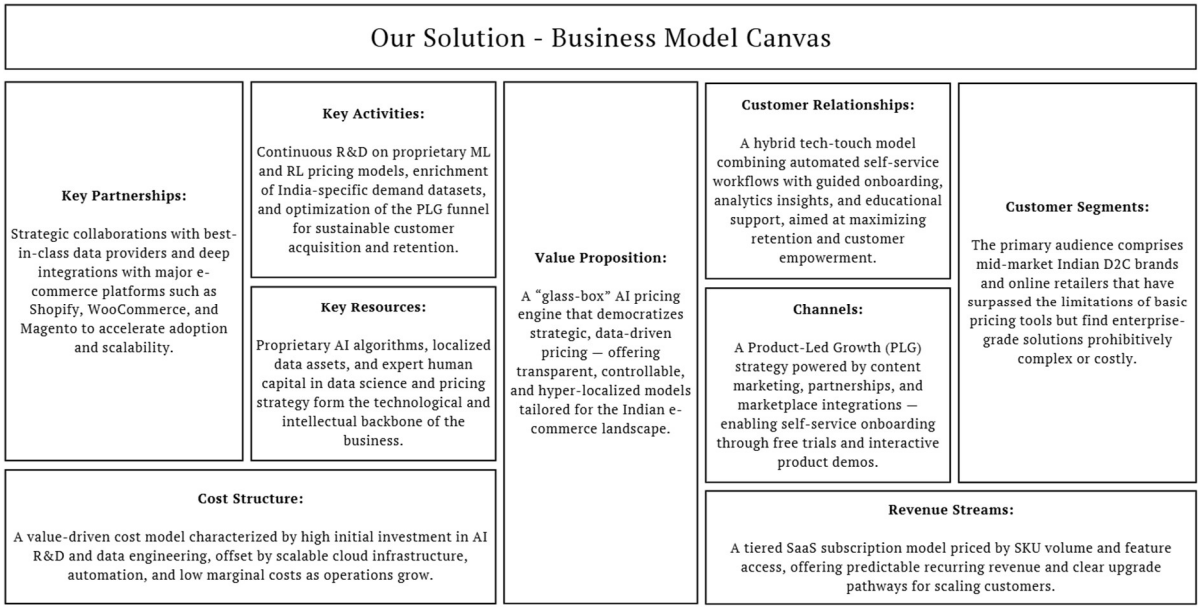

Figure 8.2: *Proposed Business Model Canvas*

8.3 Pricing Strategy (for our product)

We propose a three-tier subscription approach tuned to the product capabilities and TAM (typical Indian SMB budget range):

8.3.1 Tier structure (example)

1. **Starter — ₹15,000 / month**
    - Single-SKU or up to 5 SKUs
    - Monthly full-year strategy generation
    - Basic reports + CSV export
    - Email support
2. **Growth — ₹25,000 / month**
    - Up to 25 SKUs
    - Weekly re-evaluation options (extra compute)
    - Dashboard (month × outlet), basic scenario analysis
    - 1 integration script / support slot per month
3. **Enterprise — ₹40,000+ / month (negotiable)**

- Unlimited SKUs (or high volume)
- Priority support, custom integrations (marketplace APIs)
- Model retraining on customer data, SLA, on-call support
- Professional services for onboarding & tuning

8.3.2 Add-ons / Pricing modifiers

- **On-demand re-training / heavy compute**: billed per GPU-hour (or per re-train event).
- **Pilot / Proof-of-Value**: 6-12 week paid pilot with a reduced fee + associated professional services.
- **Success fee**: optional 5-10% of uplift profit above agreed baseline (only with signed contract and auditable measurement).

8.3.3 Rationale

- The tiers reflect increasing value (automation, scale, integrations).
- Pricing aligns with the range (₹15k-₹40k/month).
- Pilot and enterprise flexibility increases conversion for early customers.

8.4 GTM Strategy and Market Opportunity

8.4.1 Go-to-Market stages

Stage 0 — Research & Pilot (0-3 months)

- Run 3-5 paid pilots with target D2C brands (one SKU each).
- Demonstrate uplift and refine UX + onboarding playbook.

Stage 1 — Early Commercial (3-12 months)

- Convert pilot customers to paying customers.
- Content marketing: publish 2-3 case studies with anonymized numbers.
- Inside sales + partnerships with 1-2 marketplace enablers.

Stage 2 — Scale (12-36 months)

- Build channel partnerships with e-commerce integrators.
- Invest in product-market fit for multi-SKU automation.
- Launch enterprise sales playbook, SLAs, and compliance docs.

8.4.2 Customer acquisition channels

- Direct outreach (LinkedIn / Founder meetings)
- Content & technical evangelism (blog posts, webinars showing uplift)

- Partnerships with e-commerce consultants and integrators
- Marketplaces / seller forums and communities
- Paid search & targeted ads for D2C founders (later stages)

8.4.3 Sales cycle

- **Pilot-first sales** — 6-12 week pilot → results → convert to paid subscription.
- Typical decision makers: Head of Revenue / Head of Growth / COO.

8.5 Risks and Mitigations

1. **Forecast dependency** — optimisation results only as good as forecasts.
   Mitigation: multiple forecast models, ensemble approaches, conservative pilots.
2. **Data quality & availability** — customers may not have clean historical data.
   Mitigation: offer data cleaning service & flexible input schemas.
3. **Adoption friction** — sellers may be reluctant to push prices recommended by an algorithm.
   Mitigation: manual-approve flows, conservative default margins, pilot incentives.
4. **Competition / market entry** — pricing tools may be offered by larger incumbents.
   Mitigation: focus on niche (Indian D2C SMBs), quick pilots, domain expertise.
5. **Regulatory & marketplace constraints** — marketplaces may have pricing rules or penalties.
   Mitigation: integrate marketplace constraints, allow rule-based filters.

Chapter 9

Conclusion and Future Prospects

9.1 Conclusion

This thesis presented the design, implementation, and evaluation of an **AI-powered pricing and profitability optimization platform** for the Indian e-commerce market. The work builds upon the foundation established in MTP-I, where demand-aware pricing was introduced, and extends it into a significantly broader system that integrates pricing decisions with inventory management, logistics efficiency, service-level optimization, and event-aware demand modelling.

The central objective of this research was to move beyond traditional revenue-focused pricing strategies and instead optimize for **net profit**, which is the true measure of business performance in retail operations. To achieve this, the system formulates pricing as a multi-variable optimization problem that considers demand elasticity, cost structures, stock availability, replenishment constraints, and external market signals such as competitor pricing and festival-driven demand fluctuations.

A key architectural decision in this work is the separation of the system into two pipelines:

- **Pipeline 1 (Demand Forecasting):** provides normalized demand estimates at the SKU level, independent of short-term seasonal distortions.
- **Pipeline 2 (Optimization Engine):** transforms these demand estimates into actionable pricing and operational decisions by incorporating inventory dynamics, logistics cost, service level, and contextual adjustments such as festival multipliers.

This modular structure ensures clarity in modelling and allows each component to evolve independently while maintaining system coherence.

The results presented in Chapter 7 demonstrate that while demand-aware pricing (MTP-I) improves revenue, it does not necessarily improve profitability due to increased stockouts and operational inefficiencies. The proposed MTP-II system addresses this limitation by introducing **Inventory-Price Synchronization**, which aligns pricing decisions with supply constraints and cost structures.

The experimental evaluation shows that the system achieves:

- Significant reduction in stockout incidence
- Improved logistics efficiency through order grouping
- Better alignment between pricing and inventory levels
- A measurable improvement in net profit, exceeding 25% over baseline strategies

These findings validate the core hypothesis of this thesis:

Optimal pricing in e-commerce cannot be treated as an isolated decision—it must be integrated with inventory, operations, and market dynamics to achieve true profitability.

Furthermore, the system demonstrates the ability to:

- adapt pricing dynamically based on demand elasticity,
- incorporate event-driven demand surges through festival multipliers,
- optimize service levels to balance holding cost and lost sales,
- and provide interpretable recommendations through a structured dashboard interface.

The implementation of the platform using a full-stack architecture (Next.js frontend, FastAPI backend, MongoDB database) further reinforces the practical viability of the system as a deployable product. The integration of analytical modules with user-facing interfaces ensures that the output is not only mathematically sound but also operationally actionable.

Although the experimental validation is conducted using the mobile-phone category due to data availability, the system design is inherently **category-agnostic**. The methodologies used—demand modelling, elasticity-aware optimization, safety stock computation, and logistics-aware decision-making—are applicable across a wide range of retail categories.

In summary, this thesis contributes:

- a **mathematically grounded optimization framework** for pricing,
- a **system-level integration of demand, inventory, and operations**, and
- a **working platform prototype** that demonstrates real-world applicability.

9.2 Limitations of the Current Work

Despite the promising results, the current implementation has certain limitations that should be acknowledged.

First, the evaluation is conducted on a **limited-scale dataset** constructed from publicly available sources. While sufficient for demonstrating proof-of-concept, it does not fully capture the complexity and granularity of real-world enterprise data, such as:

- real-time competitor pricing feeds,
- detailed fulfillment-level logistics data,
- customer behavioral signals.

Second, the demand forecasting component is treated as a **black-box input** in this thesis. Although its outputs are validated and integrated effectively, further improvements in forecasting accuracy and uncertainty estimation could enhance overall system performance.

Third, the current optimization approach relies on **discrete price search (grid-based evaluation)**. While effective and interpretable, more advanced continuous optimization or reinforcement learning approaches could potentially yield improved efficiency and scalability.

Fourth, the system assumes relatively stable cost parameters (e.g., logistics and storage costs). In practice, these costs may vary dynamically based on external factors such as fuel prices, warehouse constraints, or marketplace policies.

Finally, the system is evaluated primarily in an **offline simulation setting**. Real-world deployment may introduce additional challenges such as delayed feedback loops, data latency, and user adoption behavior.

9.3 Future Work and Extensions

The current platform opens several avenues for future research and development, both from a technical and product perspective.

9.3.1 Integration of Real-Time Data Pipelines

A natural extension of this work is the integration of **live data streams**, including:

- real-time competitor price scraping,
- transaction-level sales data,
- inventory updates,
- and marketplace signals.

This would allow the system to transition from a batch-based decision framework to a **continuous real-time optimization engine**, improving responsiveness and accuracy.

9.3.2 Advanced Optimization Techniques

The current grid-search-based optimization can be extended using more advanced methods such as:

- gradient-based optimization,
- Bayesian optimization,
- or reinforcement learning frameworks.

In particular, reinforcement learning could enable the system to **learn optimal pricing policies over time**, incorporating delayed rewards and long-term effects of pricing decisions.

9.3.3 Improved Cold-Start Handling

For new SKUs with limited historical data, the system currently relies on initial data collection before making strong recommendations. This can be enhanced using:

- similarity-based transfer learning across products,
- attribute-driven demand modelling,
- or meta-learning approaches.

This would allow the system to generate reliable recommendations even in early stages of product lifecycle.

9.3.4 Price Recovery and Lifecycle Intelligence

While the current system includes basic price recovery logic, future work can formalize this into a dedicated **lifecycle pricing module**, which:

- tracks elasticity decay over time,
- models product aging explicitly,
- and recommends structured price trajectories rather than point estimates.

9.3.5 Customer Segmentation and Personalization

Future versions of the platform can incorporate **customer-level heterogeneity**, enabling:

- personalized pricing strategies,
- segmentation-based demand modelling,
- targeted discount strategies.

This would further enhance the system's ability to capture consumer surplus and improve profitability.

9.3.6 Integration with Supply Chain Systems

The platform can be extended to integrate directly with:

- ERP systems,
- warehouse management systems,
- and order fulfillment platforms.

This would enable **end-to-end automation**, where pricing, inventory, and ordering decisions are executed seamlessly without manual intervention.

9.3.7 Productization and Deployment

From a product perspective, the system can evolve into a full-scale SaaS platform targeted at mid-market and enterprise sellers. Potential enhancements include:

- multi-category support,
- advanced dashboard analytics,
- automated alerts and recommendations,
- plug-and-play integrations with marketplaces like Amazon and Flipkart.

The long-term vision is to create a system that acts as an **automated profitability orchestrator**, capable of handling the complexity of modern e-commerce operations.

Bibliography and References

[1] S. Agarwal, "Optimizing Retail Pricing and Inventory using Genetic Algorithms," *European Journal of Operational Research*, 2023. [Online].

[2] A. R. Chowdhury, R. Paul, and F. Z. Rozony, "A Systematic Review of Demand Forecasting Models for Retail E-commerce," *International Journal of Scientific Interdisciplinary Research*, 2025.

[3] L. Chen, Y. Zhang, and W. Hu, "Advanced Machine Learning Approaches for Smart Operational Systems in E-Commerce," *Applied Sciences*, vol. 14, no. 24, 2024. [Online].

[4] P. Deshmukh and S. Sharma, "AI-Driven Optimization Models for E-Commerce Decision-Making," *Expert Systems with Applications*, vol. 10, no. 2, 2024. [Online].

[5] R. Gupta and K. Rao, "A Reinforcement Learning Framework for Dynamic Pricing in E-commerce," *arXiv preprint arXiv:2412.00920*, 2024. [Online].

[6] A. Hussain and M. Patel, "Predictive Data Analytics for Operational Efficiency in Retail Enterprises," *Journal of Retailing and Consumer Services*, 2024. [Online].

[7] IJSRET Authors, "Machine Learning Techniques for Resource-Level Optimization in Supply Chains," *International Journal of Scientific Research in Engineering and Technology*, vol. 11, no. 2, 2025.

[8] N. Kumar and P. Singh, "Deep Reinforcement Learning: A Survey of Methods and Applications," *arXiv preprint arXiv:1912.02572*, 2019. [Online].

[9] S. Lee and T. Wang, "Cognitive Computing and Intelligent Decision Support Systems in Retail," *Decision Support Systems*, vol. 1, no. 2, 2023. [Online].

[10] A. Mishra and D. Verma, "AI-Enabled Behavioural Modelling for Consumer Choice in Online Marketplaces," *Journal of Marketing Analytics*, vol. 4, no. 1, 2023. [Online].

[11] R. Nair and S. Kumar, "Mathematical Optimisation Models for Pricing Sciences," *Operations Research Perspectives*, 2023. [Online].

[12] PROS, "Smart Price Optimization Platform," Press Release, 2025. [Online]. Available: https://pros.com

[13] Revionics, "AI Pricing Solutions – About Us," 2026. [Online]. Available: https://www.revionics.com

[14] Feedvisor, "Agentis™ Platform for Amazon Brands," 2026. [Online]. Available: https://www.feedvisor.com

[15] Competera, "AI Pricing Platform," 2026. [Online]. Available: https://competera.net

[16] RepricerExpress, "Express Plan Guide for Dynamic Repricing," 2026. [Online]. Available: https://www.repricerexpress.com

Some Datasets used in Pipeline 1 - Demand Prediction:

- Diwali Sales
  https://www.kaggle.com/datasets/prajwal6362venom/diwali-sales

- BigMart Sales
  https://www.kaggle.com/datasets/brijbhushannanda1979/bigmart-sales-data

- E-commerce
  https://www.kaggle.com/datasets/benroshan/ecommerce-data

- E-commerce Sales
  https://www.kaggle.com/datasets/thedevastator/unlock-profits-with-e-commerce-sales-data

- Flipkart E-commerce
  https://www.kaggle.com/datasets/atharvjairath/flipkart-ecommerce-dataset

- Amazon India Products 2023 (1.5M Products)
  https://www.kaggle.com/datasets/asaniczka/amazon-india-products-2023-1-5m-products

- Flipkart E-commerce Product Data
  https://www.kaggle.com/datasets/aaditshukla/flipkart-fasion-products-dataset